THE COOPER UNION FOR THE ADVANCEMENT OF SCIENCE AND ART

ALBERT NERKEN SCHOOL OF ENGINEERING

A Low-Power ASIC MCU Design Based on RISC-V RV32E for IoT Applications

By

Jonghyeok Kim

A thesis submitted in partial fulfillment of the requirements for the degree of Master of Engineering

Advisor

Professor Jabeom Koo

THE COOPER UNION FOR THE ADVANCEMENT OF SCIENCE AND ART

ALBERT NERKEN SCHOOL OF ENGINEERING

This thesis was prepared under the direction of the Candidate's Thesis Advisor and has received approval. It was submitted to the Dean of the School of Engineering and the full Faculty, and was approved as partial fulfillment of the requirements for the degree of Master of Engineering.

Barry L. Shoop 05/14/2026

Barry L. Shoop, Ph.D., P.E. Date

Dean, Albert Nerken School of Engineering

Professor Jabeom Koo, Ph.D. 05/01/2026 Date

Acknowledgement

My journey to The Cooper Union began at age eleven, when a math teacher in South Korea spoke of its rigorous curriculum. Years later, my clarinet teacher echoed these praises, describing it as one of the world's top institutions. Their influence led me to explore and ultimately graduate from this remarkable school. I firmly believe that attending this school and meeting such amazing peers and faculty is the best decision I have ever made.

I would like to express my deepest gratitude to my advisor, Professor Jabeom Koo. He has been vital to the success of my work in countless ways: encouraging me to pursue research I was initially uncertain about, sharing his robust expertise, guiding me with industrial chip design tools/techniques, and troubleshooting complex problems alongside me. Having taken his Circuit Analysis, Electronics I & II, VLSI, DVLSI, and RISC-V Architecture for IoT Applications courses, I have benefited immensely from the strong fundamentals he provided. This work would not have been possible without him.

I would also like to thank Dean Shay, who was my first instructor in Digital Logic Design during the remote learning period of COVID-19. Her guidance and passion for teaching ignited my love for learning Electrical Engineering and solidified my commitment to electrical engineering. Additionally, I thank Professors Keene, Fontaine, Marano, Hausman, Cavallaro, and Hoerning for providing me with both the foundational and specialized knowledge.

Lastly, I owe an immense amount of gratitude to my family. Their consistent support for both my education and my well-being made this journey possible. I could not have reached this milestone without their tireless efforts and sacrifices.

Abstract

Battery-powered Internet of Things (IoT) sensor nodes require microcontrollers (MCUs) that minimize power consumption during prolonged idle periods while maintaining adequate processing capability for brief sensing events. Application-specific integrated circuit (ASIC) design enables further power reduction by retaining only the necessary functionality, thereby reducing transistor count and chip area.

This thesis presents the design and verification of a low-power ASIC MCU based on the RISC-V RV32E processor, integrating a three-state PMU (Active, Sleep, Deep-sleep), an SPI flash controller, a UART controller, a GPIO controller, a boot ROM, and 8 KB SRAM with external connections to a PIR sensor, an SPI flash, and a Zigbee module. Customizable application firmware, written in C, implements the complete motion sensing cycle, including Zigbee communication, configurable timeout, and sleep/wake management. The resulting design achieves an Active mode power consumption of 31% compared to the STM32U5, one of the leading ultra-low-power MCUs in the industry, while Deep-sleep power remains higher due to the leakage characteristics of the 65nm general-purpose standard cell library and several design aspects identified for future optimization.

Contents

Chapter 1: Introduction 1

1.1 Motivation 1

1.2 Problem Statement and Objectives 2

1.3 Contribution of This Work 4

1.4 Structural Overview 5

Chapter 2: RISC-V Multicycle Core Design 7

2.1 Background on RISC-V 7

2.2 Reference CPU Design 8

2.3 Modifications from PicoRV32 9

2.3.1 Selection of the 16-Register File 10

2.3.2 Removal of Compressed Instructions 10

2.3.3 Removal of the PCPI and Multiply/Divide Units 10

2.3.4 Removal of Look-Ahead Memory Interface 11

2.3.5 Additional Simplifications 12

2.3.6 Simplification of Interrupt Handling 12

2.4 Instruction Execution and State Machine Control 14

2.4.1 The Fetch State 15

2.4.2 The LD_RS1 State 15

2.4.3 Paths from LD_RS1 15

2.4.4 The LD_RS2 State and Beyond 16

2.4.5 Next Fetch and the Trap State 16

Chapter 3: Power Management Unit (PMU) Design 18

3.1 Review of Low-Power Industrial Designs 18

3.1.1 Arm Cortex-M0+ 18

3.1.2 ESP32-S3 19

3.1.3 STM32U5 20

3.2 Proposed Three-State Power Model 20

3.2.1 Active Mode 21

3.2.2 Sleep (Count) Mode 21

3.2.3 Deep-Sleep Mode 21

3.3 Dual-Clock Domain Architecture 22

3.3.1 4 MHz Domain 22

3.3.2 32 kHz Domain 23

3.4 Clock Domain Crossing (CDC) 23

3.4.1 Double Flip-flop Synchronization 24

3.4.2 Synchronized Signals and Latency 25

3.4.3 Sleep Controller States 26

3.4.4 Wake-up Trigger 28

Chapter 4: Peripheral and System Integration 29

4.1 On-Chip Components 31

4.1.1 Memory Subsystem 31

4.1.2 Reset Synchronizer 31

4.1.3 Clock Sources 32

4.1.4 SRAM 32

4.1.5 Peripheral Communication Controllers 32

4.2 Off-Chip Interface 34

4.3 Firmware Development 37

Chapter 5: Verification and Simulation 40

5.1 Simulation Environment 40

5.2 Pre-Synthesis Simulation 43

5.4 Post-Synthesis Simulation 45

Chapter 6: Physical Implementation and Results 47

6.1 Physical Layout 47

6.2 Power Analysis 48

6.3 Timing Analysis 51

6.4 Discussion 52

Chapter 7: Conclusion and Future Work 54

7.1 Summary of Work 54

7.2 Future Enhancements 54

Appendix A: Pre-Synthesis Simulation Output 56

Appendix B: Post-Synthesis Simulation Output 62

Bibliography 68

List of Figures

1.1 Comparison of typical academic RISC-V projects versus this thesis. 3

2.1 CPU instruction execution state machine. 14

3.1 Metastability in synchronous systems 24

3.2 Sleep controller state machine operating algorithm 27

4.1 System architecture of the designed IoT chip with external device connections 29

4.2 Firmware data path from compilation to hardware execution 38

5.1 Simulation sensing cycle sequence 41

5.2 Pre-synthesis simulation waveforms during the complete sensing cycle 43

6.1 Layout of designed MCU (excluding SRAM) 48

List of Tables

4.1 System memory map 30

4.2 Off-chip interface signals 34

5.1 Pre-synthesis and post-synthesis performance comparison 40

6.1 Physical layout summary 47

6.2 Power analysis results of three different PMU states 49

6.3 Power comparison with a commercial ULP MCU 50

Chapter 1: Introduction

1.1 Motivation

The rapid growth of IoT sensors in smart cities, smart homes, and industrial areas is the main driver for this research. Many modern technologies, including humanoid robots, electric cars, and AI home systems, rely on a massive number of sensors to function. As these sectors grow, the demand for energy-efficient environmental sensors that support them grows proportionally. Reducing per-device power consumption would decrease the need for frequent battery replacements and lower the massive total energy used by billions of devices. The number of global active IoT connections is projected to grow from 18.6 billion in 2024 to 55 billion by 2035, representing a compound annual growth rate of 10% [1]. With tens of billions of devices expected to be deployed within the next decade, even modest improvements in per-device power consumption can lead to enormous aggregate energy savings.

Environmental sensors are central to these applications. A motion sensor in a smart home, for instance, can detect occupancy and signal a controller to turn lights on or off accordingly, eliminating energy wasted on unoccupied rooms. Similarly, temperature sensors at electric vehicle charging stations monitor ambient conditions and share data with the vehicle to adjust charging speed via MQTT or OCPP, slowing the charge rate to keep the battery within its safe operating window while optimizing charging time as the cells warm up [2]. Such sensor applications have become increasingly important across most industries.

Many sensors are deployed in locations that are difficult or dangerous for humans to reach. Seismometers and gas sensors are often installed inside volcanic craters or on steep slopes to provide early warnings of eruptions. Other sensors are mounted at the top of cellular towers to

monitor antenna alignment or detect ice buildup. In these cases, maximizing battery life through low-power electronics is essential, as it reduces the frequency with which technicians must access these hazardous locations for maintenance.

Additionally, reducing Deep-sleep power further could enable operation from energy harvesting sources such as small solar panels, eliminating battery dependency entirely. Many solar-powered sensors tend to suffer from intermittent connectivity because they consume more energy than their panels can generate. Improving energy efficiency at the chip level is key to closing this power gap and enabling more sustainable sensor networks.

1.2 Problem Statement and Objectives

The IoT sensors can achieve substantial energy savings by employing optimized power management and lightweight processors [3]. By integrating a streamlined RISC-V RV32E core with a custom power management unit and application-specific peripherals, this project aims to demonstrate a complete, functionally verified ASIC MCU for the smart home motion detection application.

As illustrated in Figure 1.1, most academic research on low-power RISC-V design focuses on optimizing individual components only, such as subthreshold processor cores, energy-efficient register files, or novel pipeline architectures. While these contributions advance the performance in specific modules, they rarely demonstrate a complete, application-ready system that integrates the processor with power management, peripheral controllers, firmware, and a real sensing application on a single chip. The gap between an optimized processor core and a functional IoT sensor node remains largely unaddressed in academic work. Thus, this project addresses that gap by designing a complete motion sensor IoT chip from RTL through synthesis and physical layout.

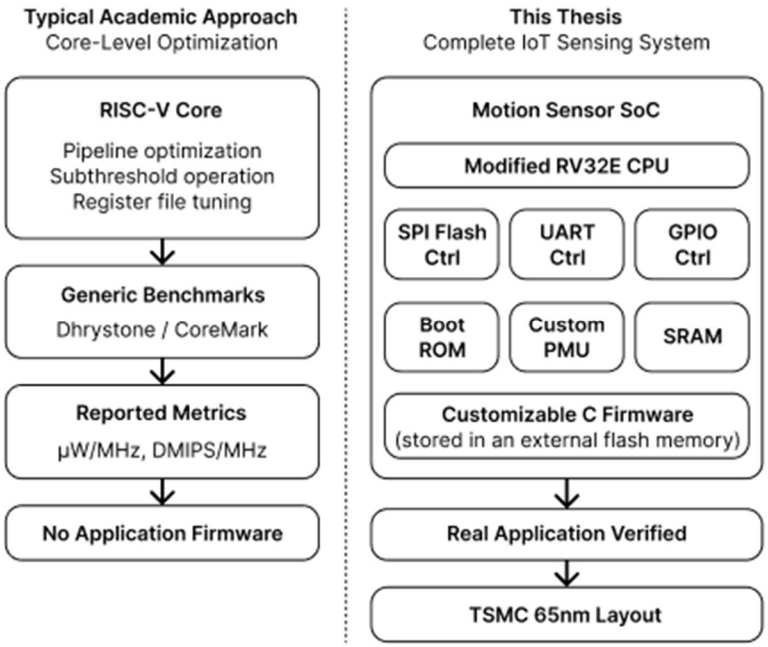


Figure 1.1: Comparison of typical academic RISC-V projects versus this thesis.

Beyond functional completeness, this project targets low-power consumption to maximize battery life in field-deployed IoT applications. Industrial research has demonstrated that ASIC-based designs can extend sensor longevity to 15 years. At this level of efficiency, the primary bottleneck seems to shift from electronic power draw to the battery's physical shelf life. While high-end Lithium Manganese Dioxide ($LiMnO^2$) batteries offer a shelf life of 10 to 15 years, the custom ASIC theoretically supports a lifespan of 23.8 years based on its power profile [4].

However, real-world factors such as temperature fluctuations, network noise, and non-linear discharge typically reduce these theoretical estimates by 25% to 50%. This disparity underscores the continual need for silicon-level power optimization to maximize the limited energy available in the field. Thus, this project aims to extend battery lifespan even further through application-specific architectural simplification and aggressive clock gating, reducing active power to only what the sensing task requires.

1.3 Contribution of This Work

This thesis presents the design, synthesis, and verification of a RISC-V RV32E motion sensor IoT chip with an integrated power management unit (PMU), targeting low-power operation on the TSMC 65nm process. Unlike most academic RISC-V projects that optimize a single module and validate with synthetic benchmarks, this work delivers a complete system-on-chip verified with real application firmware throughout the full ASIC design flow. The key contributions are as follows:

- A modified CPU architecture based on PicoRV32, configured for the RV32E embedded profile with a reduced 16-register file and extended with a Wait for Interrupt (WFI) output signal to enable the PMU to gate the CPU clock upon entering sleep modes.
- A custom PMU implementing three power states (Active, Sleep, and Deep-sleep) with a dual-clock domain architecture: a 4 MHz domain for CPU and peripheral operation and a 32 kHz domain for low-power timekeeping. The PMU manages clock gating, asynchronous PIR wake-up from Deep-sleep, and a programmable real-time clock (RTC) timeout counter, with double-flop synchronizers handling all clock domain crossings.
- A complete SoC integration comprising a RISC-V RV32E processor, SPI flash controller, UART controller, GPIO controller, boot ROM, and 8 KB SRAM, connected through a unified memory-mapped bus. Application firmware written in C implements the full motion sensing cycle: PIR detection, Zigbee light-control transmission, configurable timeout, and sleep/wake management. The firmware is designed to be application-configurable, allowing parameters such as timeout duration, sensor type, and the complete sensing cycle sequence to be modified without hardware changes.

- A two-stage verification methodology using a unified testbench for both pre-synthesis RTL simulation and post-synthesis gate-level simulation, confirming functional equivalence between the behavioral RTL and the synthesized TSMC 65nm netlist.
- Synthesis using Synopsys Design Compiler, physical layout using Synopsys IC Compiler II on the TSMC 65nm process, and power analysis across all three PMU states to evaluate Active, Sleep, and Deep-sleep power consumption.

This work covers the entire ASIC design process, from CPU customization and RTL design to synthesis, gate-level verification, and physical layout of the low-power MCU.

1.4 Structural Overview

Chapter 2 provides a background on the RISC-V RV32E architecture and explains why it was chosen for its small size and power efficiency. It also details the modifications made to the PicoRV32 reference design to optimize it for this project. Chapter 3 discusses the Power Management Unit design in detail, including the three-state power model and the dual-clock domain architecture. It explains the methods used to manage signals between different clock domains to ensure reliable wake-up operation.

Chapter 4 describes the integration of the processor with on-chip components such as SRAM and peripheral controllers, as well as off-chip components including external SPI flash and the PIR sensor. It also presents the firmware development process and the memory-mapped peripheral access model. Chapter 5 covers the verification methodology, presenting the unified testbench, pre-synthesis RTL simulation results, the synthesis process, and post-synthesis gate-level simulation.

Chapter 6 presents the physical implementation results, including the ICC2 place-and-route layout, power analysis across the three PMU states with comparison to commercial MCUs, and pre-layout timing analysis. Chapter 7 summarizes the conclusions of the work and proposes future enhancements, including macro-compiled SRAM integration and stricter power gating to reduce leakage.

Chapter 2: RISC-V Multicycle Core Design

2.1 Background on RISC-V

The ARM architecture is widely considered a standard for energy-efficient design. However, it is a proprietary architecture [5], which means it requires expensive licensing fees and does not allow designers to modify the CPU core to meet specific project needs. On the other hand, the RISC-V architecture is an open-source standard that allows designers to freely customize the CPU without paying licensing fees [5]. This freedom is vital for specialized research like this thesis, as it allows the removal of unnecessary features to save power or the addition of custom instructions for specific tasks.

RISC-V was originally developed at the University of California, Berkeley with the objective of providing a simple, modular, and extensible ISA [5]. Unlike traditional architectures that have developed over time with considerable legacy limitations, RISC-V follows a clean-slate design philosophy. The ISA comprises a minimal base instruction set and a set of optional standard extensions, including integer multiplication and division, atomic operations, and floating-point support [6]. This modular structure allows designers to include only the features necessary for a given application, thereby reducing hardware complexity, silicon area, and power consumption.

Both ARM and RISC-V are based on the Reduced Instruction Set Computer (RISC) paradigm, which emphasizes simple instructions, a load/store architecture, and efficient hardware implementation [6]. In RISC-based systems, arithmetic and logical operations are performed primarily on register operands, while memory access is limited to explicit load and store instructions. This approach simplifies processor design and improves timing predictability, making it well-suited for embedded and real-time applications.

To support a wide range of applications, RISC-V defines several base integer variants, including RV32I, RV64I, and RV128I, each corresponding to a different data path width. For highly resource-constrained systems, the RV32E subset reduces the number of general-purpose registers from 32 to 16 [6]. This reduction significantly reduces the register file size, resulting in lower silicon area and reduced power consumption. Such characteristics make RV32E particularly suitable for Internet of Things (IoT) and low-power embedded applications.

Another major benefit of RISC-V is its expanding software ecosystem. The ISA is supported by widely adopted compiler toolchains such as GCC and LLVM, which enable compilation of high-level programming languages (e.g., C, C++) for RISC-V targets. [6, 7, 8]. Furthermore, the availability of open-source processor cores, simulators, and verification tools makes development faster and lowers barriers for both academic and industrial hardware designers.

Finally, the open and well-documented nature of the RISC-V specification makes it particularly suitable for education and architectural research. Designers can customize all aspects of the system, from instruction encoding to microarchitectural implementation, without being limited by proprietary platforms. This level of transparency promotes innovation and supports the development of highly optimized, application-specific processor designs.

2.2 Reference CPU Design

PicoRV32 is an open-source, size-optimized processor core implementing the RISC-V instruction set architecture, developed by Claire Xenia Wolf and distributed under the permissive ISC license [9]. PicoRV32 specifically targets applications where silicon area and design simplicity take precedence over raw performance, making it well-suited for implementation on field-

programmable gate arrays (FPGAs) and application-specific integrated circuits (ASICs) in resource-constrained embedded systems [9].

The processor employs a multicycle execution model in which each instruction is decomposed into a sequence of micro-operations executed over multiple clock cycles. Although multicycle designs have lower instructions-per-cycle (IPC) than pipelined architectures, they provide several key benefits for embedded designs. These include a smaller silicon footprint resulting from resource sharing, a much simpler control unit, and deterministic timing. The original PicoRV32 implementation offers extensive configurability through Verilog parameters, allowing designers to selectively enable features such as hardware multiplication and division units, compressed instruction support, interrupt handling mechanisms, co-processor interfaces, debugging interfaces, and timing optimizations [9]. This allows the processor to be tailored to specific application requirements.

2.3 Modifications from PicoRV32

The modifications described below have been made to transform the original PicoRV32 from a highly configurable, general-purpose RISC-V implementation into a streamlined processor core optimized for minimal resource utilization. By removing unnecessary features and simplifying complex configuration logic, the implementation was reduced from approximately 3,050 lines of Verilog to roughly 1,000 lines. While these changes limit performance at higher clock speeds, the resulting architecture is exceptionally well-suited for IoT applications where silicon area and power consumption are the primary concerns. Although the design might not be suitable for applications that demand complex calculations, such as multiplication or division, it is intended for simple sensing and sending packets to the server application.

2.3.1 Selection of the 16-Register File

The RV32E base integer instruction set was selected for this implementation to prioritize silicon efficiency. By adopting this subset, all logic and physical registers associated with indices x16 through x31 were eliminated. Correspondingly, the decoder and register access logic were updated to utilize a 4-bit index rather than the standard 5-bit width. This modification results in a notable reduction in total chip size. While halving the register count can technically increase register pressure, the IoT sensing tasks in this project are lightweight enough that 16 registers are more than sufficient.

2.3.2 Removal of Compressed Instructions

The RISC-V C extension encodes common 32-bit instructions as 16-bit equivalents, typically compressing 50–60% of a program's instructions and reducing code size by 25–30% [6]. However, the C extension requires the decoder to distinguish between 16-bit and 32-bit instruction formats and expand compressed instructions before execution, adding combinational logic to the decode stage.

By restricting the ISA to 32-bit instructions alone, every instruction is uniformly 4 bytes in length and naturally word-aligned, which simplifies the fetch and decode stages and eliminates expansion logic. Since the firmware is small enough to fit within the 8 KB SRAM without compression, the area and power savings from removing decoder complexity were prioritized over code density improvements.

2.3.3 Removal of the PCPI and Multiply/Divide Units

The Pico Co-Processor Interface (PCPI), which allows for the addition of custom hardware accelerators, was excluded from this implementation. While the PCPI offers a standardized

protocol for extending functionality, it adds unnecessary overhead for the general-purpose tasks needed in this project. The targeted primary workload, which is merely reading sensor values and transmitting data via Zigbee, does not require hardware acceleration. Removing the PCPI eliminated complex handshaking logic, instruction forwarding, and result multiplexing.

To further optimize for size, the dedicated hardware units for multiplication and division were also removed. While the base PicoRV32 core supports various arithmetic extensions, these units introduce significant hardware complexity and increase the transistor count. Since the target application does not involve intensive numerical computation, these hardware units were eliminated. Testing confirmed that the firmware remains fully functional without them, since the target application does not require multiplication or division.

2.3.4 Removal of Look-Ahead Memory Interface

The look-ahead memory interface found in the original PicoRV32, including signals such as mem_la_read and mem_la_addr, was removed to simplify the processor's external connections. In high-performance systems, these signals act as a heads-up, notifying the memory what is coming before the actual request is sent. This allows complex memory systems to start decoding addresses or checking caches early, saving processing time.

However, for this project, such complexity is unnecessary. Instead, a straightforward valid/ready handshaking protocol was implemented. In this setup, the processor signals that it has valid data (mem_valid), and the memory simply responds when it is ready (mem_ready). This approach is much easier to implement and works perfectly with the simple, synchronous memory used in this design.

2.3.5 Additional Simplifications

To reduce implementation complexity, several additional features have been removed. The trace interface, previously used for real-time visibility into instruction execution for debugging and performance analysis, has been replaced with minimal debug output that reports only the current program counter value. The 64-bit cycle and instruction-retired counters, which offer an extended counting range for long-running performance measurements, have been omitted in favor of 32-bit counters that are sufficient for most embedded applications.

Configuration options for timing optimizations, such as the barrel shifter, two-cycle arithmetic logic unit (ALU) operations, and two-cycle comparison operations, have been eliminated to maintain a consistent multicycle execution model while minimizing hardware logic. Finally, the bus interface wrapper modules for the AXI4-Lite and Wishbone protocols included in the original PicoRV32 distribution are not included in this implementation, as the target system uses a custom memory interface architecture.

2.3.6 Simplification of Interrupt Handling

The interrupt handling system has been simplified to its core components to prioritize area efficiency and low-power handling. Features present in the original core, including hardware timers, shadow registers for rapid context preservation, and custom Q-register instructions (getq, setq, retirq, waitirq), were eliminated. Removing these features helps the design avoid the significant silicon area required for a comprehensive interrupt handler.

A major architectural change was the transition from custom, non-standard opcodes to standard RISC-V Privileged Architecture encodings. Whereas the original core employs a custom0 opcode space for interrupt instructions, this implementation utilizes the standard Wait for Interrupt

(WFI) and Machine Return from Trap (MRET) instructions. Adherence to standard encodings ensures full compatibility with off-the-shelf RISC-V toolchains and standard RISC-V assemblers or compilers.

The WFI instruction is particularly critical for the Zigbee-based sensor node. When the CPU decodes a WFI, it asserts an external signal (do_wfi), which the PMU uses to gate the CPU clock and transition to the power state configured by the firmware. If an interrupt is pending when WFI is executed, the processor treats it as a no-operation (NOP) to prevent the system from hanging. When the PMU re-enables the CPU clock upon detecting a wake event, execution resumes from the instruction immediately following WFI.

2.4 Instruction Execution and State Machine Control

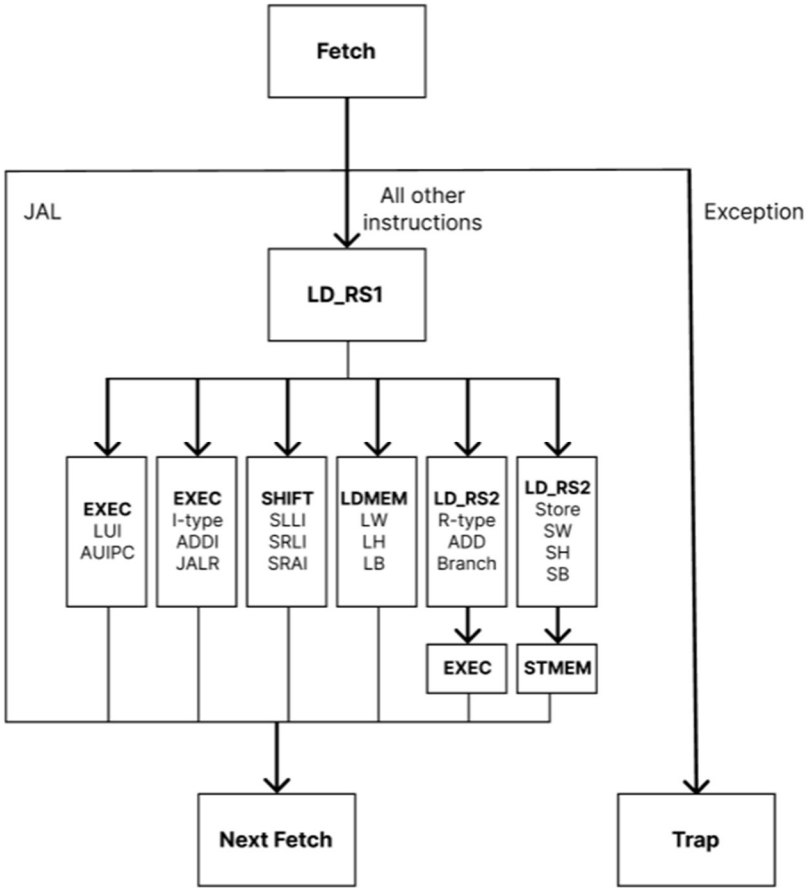


Figure 2.1: CPU instruction execution state machine.

Figure 2.1 illustrates the state machine that controls instruction execution in the modified multicycle RISC-V RV32E processor. This multicycle design breaks instruction execution into multiple states. This approach enables sharing other hardware components, such as the ALU and memory interface, across multiple phases, thereby reducing overall chip area at the expense of requiring multiple cycles per instruction.

2.4.1 The Fetch State

Every instruction begins in the Fetch state. In this state, the CPU sends the current program counter (PC) value to memory and waits for the instruction data to return. Once the memory responds with the instruction word, the CPU latches it into the instruction register and begins decoding the opcode.

The Fetch state also finalizes the write-back of the previous instruction's result to the register file. This overlap allows the CPU to be more efficient by combining the end of one instruction with the beginning of the next. From Fetch, most instructions proceed to LD_RS1, but there are two special cases. The Jump and Link (JAL) instruction completes entirely within the Fetch state and loops back immediately. If an exception or illegal instruction is detected early, the CPU transitions directly to the Trap state.

2.4.2 The LD_RS1 State

The Load Register Source 1 (LD_RS1) state reads the first source register (rs1) from the register file and loads its value into the internal working register (reg_op1). During this state, the CPU also finishes decoding the instruction and determines which path to take next. Instructions that only need one register operand (or none) can skip ahead, while instructions requiring two register operands advance to LD_RS2.

2.4.3 Paths from LD_RS1

From LD_RS1, the CPU follows one of six possible paths based on the instruction type. LUI and AUIPC proceed directly to the EXEC state, as they do not require register values. LUI uses zero, and AUIPC uses the PC as the first operand, combined with the immediate value. I-type ALU instructions such as ADDI, XORI, and JALR also proceed directly to EXEC because they only need one register operand (rs1). The second operand is the immediate value encoded in the

instruction itself. Shift-immediate instructions (SLLI, SRLI, SRAI) take a path to SHIFT, which handles multi-cycle shift operations by shifting a few bits at a time. Load instructions (LW, LH, LB, and their unsigned variants) proceed to LDMEM to calculate the effective address and access memory. R-type instructions (ADD, SUB, AND, OR, XOR, etc.) and branch instructions require both rs1 and rs2, so they must go through LD_RS2 first. Store instructions (SW, SH, SB) also require LD_RS2 to access both the base address and data registers.

2.4.4 The LD_RS2 State and Beyond

Instructions that reach LD_RS2 read the second source register (rs2) from the register file and write it to reg_op2. Subsequently, R-type ALU and branch instructions proceed to EXEC, where computation or comparison occurs. Store instructions take a different path to STMEM, which requires three cycles to calculate the effective address, issue the memory write, and complete the transfer. The EXEC state handles arithmetic and logical operations, and for branch instructions, evaluates the branch condition and updates the PC if the branch is taken.

2.4.5 Next Fetch and the Trap State

All instruction paths eventually converge back to the Fetch state to begin the next instruction. Figure 2.1 shows this convergence with all paths leading to Next Fetch at the bottom. The number of cycles required varies by instruction type:

- JAL: 3 cycles (FETCH → FETCH → FETCH)
- I-Type ALU: 5 cycles (FETCH → FETCH → LD_RS1 → EXEC → FETCH)
- LUI/AUIPC: 5 cycles (FETCH → FETCH → LD_RS1 → EXEC → FETCH)
- JALR: 5 cycles (FETCH → FETCH → LD_RS1 → EXEC → FETCH)

- R-Type ALU: 6 cycles (FETCH → FETCH → LD_RS1 → LD_RS2 → EXEC → FETCH)
- Branch: 6 cycles (FETCH → FETCH → LD_RS1 → LD_RS2 → EXEC → FETCH)
- Load: 7 cycles (FETCH → FETCH → LD_RS1 → LDMEM(×3) → FETCH)
- Store: 8 cycles (FETCH → FETCH → LD_RS1 → LD_RS2 → STMEM(×3) → FETCH)
- Shift Imm: 5-9 cycles (depends on shift amount, using TWO_STAGE_SHIFT)

The cycle counts shown are minimums assuming ideal single-cycle memory latency. In real hardware, both instruction fetches and data accesses will cause additional wait cycles depending on the memory subsystem's response time. The Trap state exists as an exception handler for illegal instructions. Once the CPU enters the Trap state, it asserts the trap output signal and halts until the system is reset. Note that the same trap output is also asserted when the CPU executes WFI, as both conditions indicate the CPU has stopped executing instructions. The PMU distinguishes between these cases through the do_wfi signal.

Chapter 3: Power Management Unit (PMU) Design

3.1 Review of Low-Power Industrial Designs

One commercial low-power processor and two microcontroller architectures were reviewed to provide context for established power management approaches in the industry. The Arm Cortex-M0+ emphasizes minimizing area and gate count in 32-bit processors. The ESP32-S3 is designed for IoT applications that demand wireless connectivity and high processing speed, while the STM32U5 is widely recognized for its sophisticated ultra-low-power management across multiple sleep modes.

These three designs cover a wide range of power performance tradeoffs and architectural complexity. Understanding how each handles sleep mode entry, clock control, and wake-up detection informed the decision in this work to pursue a simpler architecture built around clock gating rather than techniques such as multi-mode power domains, hardware multipliers, or dedicated co-processors.

3.1.1 Arm Cortex-M0+

The Arm Cortex-M0+ processor is widely regarded as the industry benchmark for its area-efficiency operating on a 32-bit ISA. As detailed in its datasheet v2.1, the architecture utilizes a two-stage pipeline to minimize logic gate overhead and reduce switching activity during branch operations [10].

Low-power sleep mode entry is managed via two specialized architectural states: Wait for Interrupt (WFI) and Wait for Event (WFE) [10]. WFI halts the clock signal to the core while maintaining responsiveness to external peripheral interrupts. Conversely, WFE suspends execution

until a specific trigger occurs, allowing the processor to remain dormant for longer durations to maximize energy savings [10].

Furthermore, the implementation includes a hardware single-cycle (32x32) multiplier, which enables the processor to complete complex arithmetic operations in a single clock cycle [10]. This increases performance throughput and reduces the total energy per instruction (EPI) for math-intensive processing of sensor data.

According to the Cortex-M0+ datasheet, it achieves dynamic power consumption as low as 3.8 μW/MHz (typically 5.4 μW/MHz) when synthesized in a 40nm LP process (9-track, typical 1.1V, 25°C) [10]. At 125°C, its clock frequency can reach 297 MHz [10]. The Cortex-M0+ is an impressive low-power processor, but it includes features such as a hardware multiplier and a two-stage pipeline that represent more computational overhead than a simple sensor node requires.

3.1.2 ESP32-S3

The ESP32-S3 is one of the most widely used microcontrollers in IoT applications, particularly in educational and prototyping environments. For high-performance IoT applications that require wireless connectivity, the ESP32-S3 employs dual-processor architecture. An internal ultra-low-power (ULP) Co-processor remains powered during Deep-sleep mode, allowing programs stored in RTC memory to access RTC peripherals while the main cores are suspended [11].

This co-processor is specifically tasked with sensor polling and basic logic maintenance. While the main cores support clock speeds up to 240 MHz, this high-performance design results in a current draw of approximately 107.9 mA in Active mode [11], which is considered high compared to other industrial low-power MCUs. It can reduce power consumption to 7 μA during

Deep-sleep mode. While its dual-core setup and integrated Wi-Fi/Bluetooth support heavy multitasking, they result in considerably higher power consumption [11].

3.1.3 STM32U5

The STM32U5 MCU series is considered an industry benchmark for advanced energy management. While the Cortex-M0+ focuses on a minimal silicon footprint, the STM32U5 (based on the Arm Cortex-M33) targets ULP applications through sophisticated power-down modes and autonomous peripherals [12].

This architecture uses an external interrupt controller (EXTI) to monitor GPIO pins and generate wake-up events [12]. In its lowest-power modes, the system utilizes a lower clock frequency, such as 32 kHz [12]. The STM32U5 consumes 160 nA in Shutdown mode and 19.5 µA/MHz in Active (Run) mode [12]. It includes various power modes, including Shutdown, Standby, Stop 1/2/3, and Run modes [12]. However, achieving this efficiency requires several peripheral blocks, such as the EXTI, to remain powered at all times.

3.2 Proposed Three-State Power Model

The proposed system architecture is specifically designed for low-power edge nodes that perform periodic environmental sensing and data transmission via wireless protocols. For the RTL simulation performed in this project, Zigbee was utilized as the target protocol. To achieve a minimal silicon footprint and maximize battery longevity, the power management unit (PMU) implements three simplified power states: Active, Sleep, and Deep-sleep modes. This simplified approach reduces the total logic gate count while preserving the core functionality required for sensor nodes.

3.2.1 Active Mode

The Active mode is the primary execution mode in which the CPU runs firmware from SRAM with both the 4 MHz system clock and 32 kHz RTC clock active. It is used during the initial boot (power-on reset) to copy firmware from the SPI flash to SRAM, process sensor events to send an RF packet to signal the event, and handle timeouts to send an RF packet indicating that the event has ended. Note that this MCU has been validated only with sensors that do not require computational processing during data acquisition. For example, passive infrared (PIR) sensors (e.g., EKMC1601111) do not require computation to sense motion and set the output port high. Once the CPU completes its task, it executes a WFI instruction, which signals the PMU to clock-gate the CPU and transition to the appropriate low-power mode.

3.2.2 Sleep (Count) Mode

The Sleep mode is a low-power state that runs a programmable counter while still responding to sensor interrupts without requiring CPU activity. During this mode, the CPU clock is gated off to eliminate dynamic power consumption, while the 32 kHz clock remains active for the timeout counter. In a production implementation, the 4 MHz oscillator would also be disabled entirely through the PMU's oscillator enable output.

The timeout duration is determined by firmware parameters stored in flash memory, allowing for field-adjustable timing. For instance, if the PIR sensor detects no subsequent motion within the predefined period, the PMU briefly reactivates the core to transmit a termination (e.g., lights-off) signal before moving the system into its lowest energy state.

3.2.3 Deep-Sleep Mode

The Deep-sleep mode represents the system's baseline power state. In this mode, the CPU clock is gated off, and no timekeeping is required. Both oscillators would be disabled through the PMU's

oscillator enable outputs, leaving only the asynchronous PIR wake path active. The system maintains this state until an asynchronous interrupt, such as a PIR sensor trigger, is detected.

To respond to external interrupt signals, the PMU is designed with asynchronous detection logic. This allows the system to recognize a hardware trigger and initiate the wake-up sequence to transition back into Active mode without requiring an active clock.

3.3 Dual-Clock Domain Architecture

The PMU employs a dual-clock domain architecture to achieve optimal power efficiency while maintaining system responsiveness. This architecture comprises two distinct clock domains: a 4 MHz domain for high-performance operation and a 32 kHz domain for low-power timeout counter monitoring. By selectively enabling each clock domain based on the current power state, the system minimizes dynamic power consumption.

3.3.1 4 MHz Domain

The 4 MHz clock domain encompasses the main state machine, CPU register interface, interrupt aggregation, clock gating control, and wake handler. The main state machine controls power mode transitions, determining when the system enters Active, Sleep, or Deep-sleep mode based on firmware commands and wake events. The CPU register interface handles all PMU register accesses from the CPU, providing consistent timing for firmware interaction.

The interrupt aggregation logic collects interrupt sources, including PIR sensor events and timeout expirations, and sends signals to the CPU through the interrupt pending register. Clock gating control manages the CPU clock enable signal, using a negative-edge latch to prevent glitches when enabling or disabling the clock. The wake handler processes wake requests from both the 32 kHz domain and the asynchronous PIR sensor input, enabling the system to transition

from sleep modes back to Active mode. The 4 MHz domain is the primary source of dynamic power consumption. During Sleep mode, only the PMU and the timeout counter remain clocked at 4 MHz while the CPU clock is gated. During Deep Sleep, the 4 MHz domain is inactive, with only the asynchronous PIR wake path remaining functional.

3.3.2 32 kHz Domain

The 32 kHz clock domain provides timekeeping and wake-monitoring capabilities with minimal power consumption. Operating approximately 125 times slower than the main clock, this domain consumes significantly less dynamic power. The sleep controller finite state machine (FSM) monitors wake signals while the CPU is in Sleep mode, transitioning through four states (SLEEP_IDLE, SLEEP_MONITOR, SLEEP_WAKE, and SLEEP_HANDOFF) to coordinate with the 4 MHz domain.

The RTC counter generates 1 Hz ticks from the 32 kHz input using a 15-bit divider, providing a stable timebase for the timeout counter. This counter enables functions such as timed wake-up for periodic sensor polling by incrementing on each RTC tick and signaling when the count reaches the firmware-configured timeout value. Wake-request generation logic latches valid wake events from either PIR detection or timeout expiration, holding the request until it is acknowledged by the 4 MHz domain. For stable signal integrity, the two clock domains communicate via two-stage synchronizers that prevent metastability during clock domain crossings. The 32 kHz domain is only enabled during Sleep mode when a timer is required.

3.4 Clock Domain Crossing (CDC)

Communication between the 4 MHz and 32 kHz domains requires synchronization to prevent metastability and ensure reliable signal transfer. The PMU employs double flip-flop synchronizers

on all cross-domain signals, introducing a latency of two to three destination clock cycles. A dedicated sleep controller running at 32 kHz manages wake event detection and coordinates state transitions with the 4 MHz domain via a handshake protocol. In Deep-Sleep mode, where no clock signal is present, a separate combinational wake path bypasses the synchronizers to restart the system directly from the PIR sensor input.

3.4.1 Double Flip-flop Synchronization

Metastability occurs when an asynchronous input transition violates a flip-flop's setup or hold time [13], resulting in an unstable output state that can affect the MCU's functionality, as shown in Figure 3.1(a).

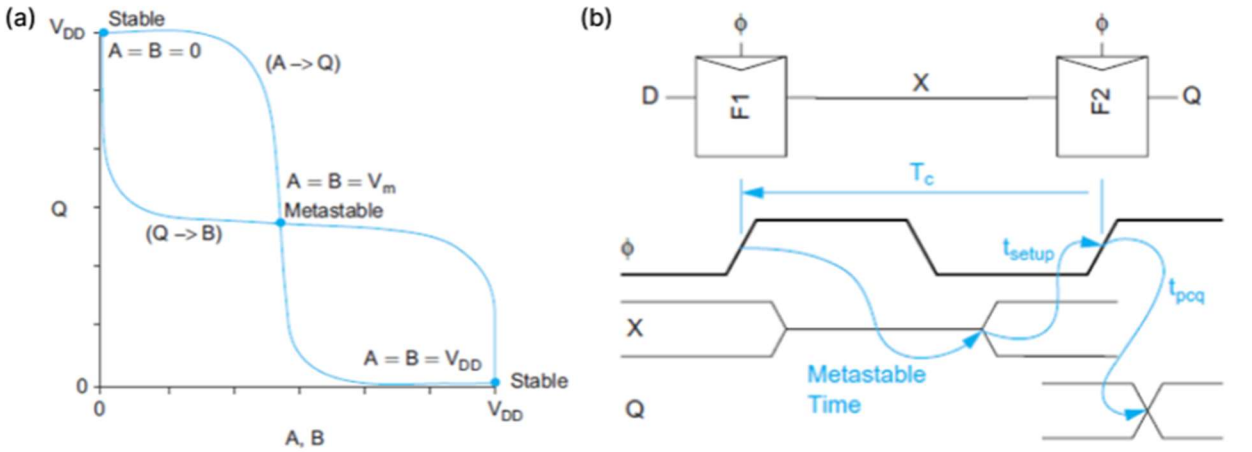


Figure 3.1: Metastability in synchronous systems. (a) Metastable state in a static latch. (b) Double synchronization method. Adapted from [13].

Since the two clock domains operate from independent crystal oscillators with no fixed phase relationship, any signal crossing between them risks violating setup or hold timing constraints at the destination flip-flop, posing a risk of metastability. Dual-stage synchronizers were placed on all signals crossing between the 4 MHz and 32 kHz domains to mitigate this risk. The first flip-flop may enter a metastable state, but the second flip-flop provides one full

destination clock period for the output to resolve to a valid logic level before it reaches the downstream state machine as shown in Figure 3.1(b) [13].

During the transition from Active to Sleep mode, the firmware sets the timeout duration and enables the 32 kHz oscillator before executing the WFI instruction. At this moment, the 4 MHz clock remains active, which results in both clock domains operating simultaneously for a brief interval. This overlap enables the dual-stage synchronizers to function, as they require active clock edges on both sides to transfer values across domains. By the time the CPU executes WFI and its clock is gated, the synchronizer has received sufficient 32 kHz clock edges to fully propagate all configuration values into the 32 kHz domain, thereby ensuring that the timeout counter receives the correct duration, and the sleep controller monitors the correct wake source.

While metastability is a physical phenomenon that cannot be observed in digital simulation, the dual-stage synchronizer reduces the probability of metastable propagation to negligible levels, following standard clock domain crossing methodology [13]. Without synchronization, a metastable value reaching the PMU state machine could cause it to enter an illegal state or leave the clock gate in an inconsistent state, potentially rendering the chip unresponsive until a power cycle.

3.4.2 Synchronized Signals and Latency

Synchronization is essential for processing signals across distinct frequency domains. By aligning asynchronous signals with the destination clock, the system prevents metastability, though this process introduces a deterministic but negligible latency.

Dual-stage synchronizers were implemented to transfer signals from the 4 MHz domain into the 32 kHz domain. These signals include the wake acknowledgment flag, the current power-

state status, and the PIR enable bit. The synchronization latency t_{sync} is determined by the destination clock period:

$$t_{sync} = NT_{dst} \tag{3.1}$$

where N is the number of synchronizer stages (2 to 3 cycles, accounting for metastability resolution and sampling uncertainty), and T_{dst} is the period of the destination clock. For the 4 MHz to 32 kHz direction:

$$t_{sync} = (2\ to\ 3) \times \frac{1}{32{,}768} = 61\ \mu s \text{ to } 92\ \mu s$$

This delay is negligible relative to the RF packet transmission that follows, which takes on the order of milliseconds. This synchronization occurs during the transition from Active to Sleep mode.

Conversely, signals originating in the 32 kHz domain, such as the PIR and timeout wake request flags, are synchronized to the 4 MHz clock. Applying the same equation with the faster destination clock:

$$t_{sync} = (2\ to\ 3) \times \frac{1}{4{,}000{,}000} = 0.5\ \mu s \text{ to } 0.75\ \mu s$$

This latency is even smaller and negligible relative to the overall system response.

3.4.3 Sleep Controller States

Running on the 32 kHz clock, the sleep controller operates in four states to manage wake detection and handshaking with the 4 MHz domain: SLEEP_IDLE, SLEEP_MONITOR, SLEEP_WAKE, and SLEEP_HANDOFF, as shown in Figure 3.2. This controller detects wake events and coordinates with the 4 MHz domain via a handshake protocol to manage power state transitions.

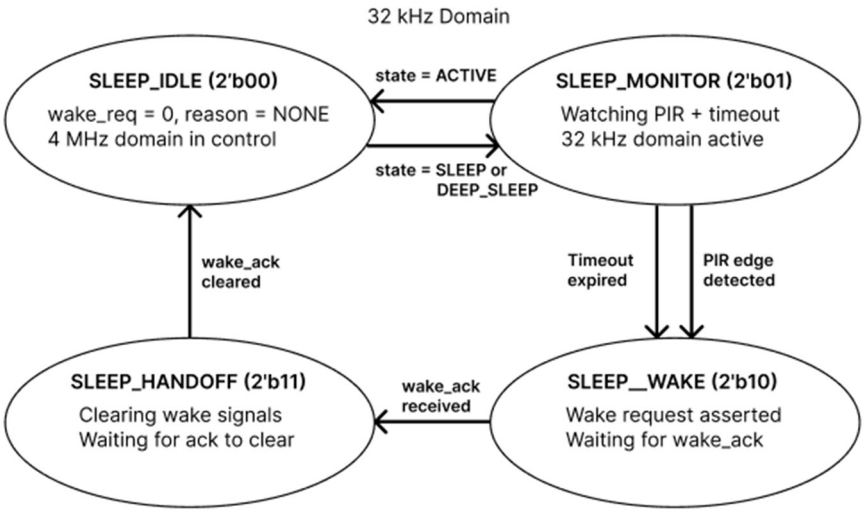


Figure 3.2: Sleep controller state machine operating algorithm.

In SLEEP_IDLE, the 4 MHz state machine handles all system operations, so the sleep controller is inactive. Wake request signals are deasserted, and the reason register is cleared to WAKE_NONE. The controller exits SLEEP_IDLE when it detects that the system has entered Sleep mode.

When the system enters Sleep mode, the sleep controller transitions to SLEEP_MONITOR and begins monitoring wake sources. Two wake sources are observed in this state: PIR sensor edges and timeout counter expiration. The controller remains in SLEEP_MONITOR until one of these wake sources takes place.

When a wake event is detected, the sleep controller enters SLEEP_WAKE and signals the 4 MHz domain. The wake reason indicates whether the event originated from PIR detection or timeout expiration. As this information is synchronized to the 4 MHz domain, the firmware recognizes this event and determines the appropriate response. Additionally, if a PIR event occurs while a timeout wake is pending, the controller updates the wake reason to PIR, as motion detection

has higher priority than a scheduled timeout. The controller holds the wake request asserted until it receives acknowledgment from the 4 MHz domain.

SLEEP_HANDOFF completes the handshake process between the two clock domains. After the synchronized wake acknowledgment has been received, the sleep controller enters this state and clears its wake request signals to prevent spurious wake events. The controller remains in SLEEP_HANDOFF until the acknowledgment signal clears, signaling that the system is now in Active mode. After the acknowledgment clears, the controller returns to SLEEP_IDLE.

3.4.4 Wake-up Trigger

The PIR sensor wake trigger is implemented for the system to respond to motion events. When motion is detected, the PIR sensor sets its output high, which is then recognized by the PMU. In Sleep mode, the PIR signal passes through a two-stage synchronizer into the 32 kHz domain to prevent metastability.

In Deep-sleep mode, the 32 kHz oscillator is disabled, so the synchronizer path is unavailable. Instead, the PMU implements a combinational wake path that directly monitors the PIR input. This asynchronous path opens the CPU clock gate using irq_pir && !wake_ack logic, allowing the 4 MHz clock to reach the PMU state machine and begin the wake-up sequence. The wake condition evaluates to true when the PIR signal is active, and no wake acknowledgment is pending, ensuring that only new motion events trigger the oscillator to restart.

Chapter 4: Peripheral and System Integration

The implementation includes both on-chip and off-chip components to function as a complete MCU for IoT applications. The on-chip components include the RISC-V CPU, PMU, boot ROM, SPI flash controller, UART controller, GPIO controller, and 8 KB SRAM. The off-chip components consist of an SPI flash for firmware storage, a Zigbee end device module for wireless communication, a PIR motion sensor, and two crystal oscillators (4 MHz and 32 kHz) for clock generation.

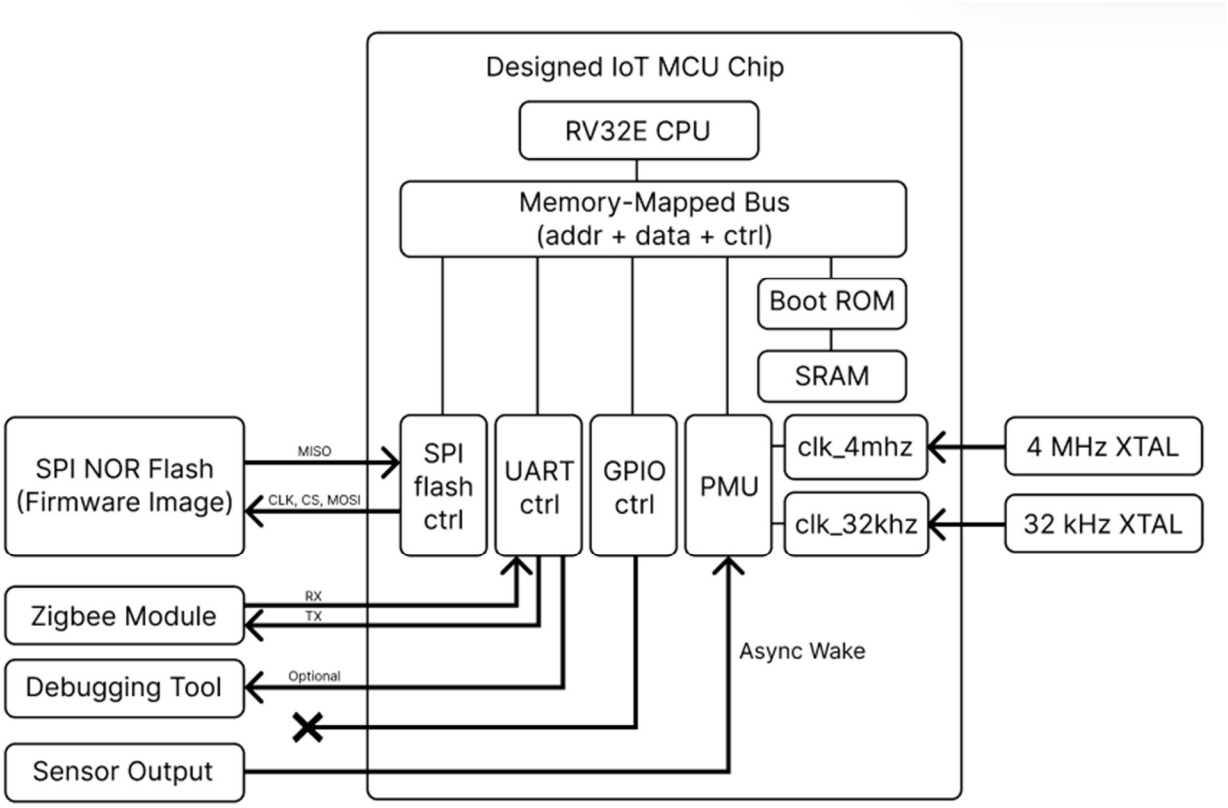


Figure 4.1: System architecture of the designed IoT chip with external device connections.

Each peripheral is memory-mapped into the processor's address space through a unified bus architecture, allowing firmware to access them via simple load and store instructions. Figure 4.1 illustrates the system architecture and external device connections.

Address Range	Peripheral	Access	Description
0x0000_0000 – 0x0000_003F	Boot ROM	Read-only	64-byte hardwired firmware copy loop
0x0800_0000 – 0x0800_FFFF	SPI Flash	Read-only	External flash via SPI
0x1000_0000 – 0x1000_00FF	UART Ctrl	Read/Write	Serial TX/RX, status, and control registers
0x1000_0200 – 0x1000_02FF	PMU	Read/Write	Power management unit registers
0x1000_0300 – 0x1000_03FF	GPIO Ctrl	Read/Write	8-bit bidirectional GPIO with direction control
0x1000_0400 – 0x1000_04FF	SPI Ctrl	Read/Write	SPI flash controller configuration registers
0x2000_0000 – 0x2000_1FFF	SRAM	Read/Write	8 KB on-chip SRAM

Table 4.1: System memory map.

The processor accesses all on-chip peripherals and memory through a unified 32-bit address space. A combinational address decoder in the top-level module examines the upper bits of each memory access and generates a chip select signal for the corresponding peripheral. For example, addresses beginning with 0x0800 activate the SPI flash controller, while addresses beginning with 0x1000_00 select the UART controller. Table 4.1 summarizes the complete memory map of the system.

4.1 On-Chip Components

The motion sensor system integrates several on-chip components to form a complete embedded system. The RISC-V processor core is supported by memory, power management, and peripheral interfaces, which are implemented on the same chip.

The CPU and PMU are the core components of this design. The CPU accesses PMU configuration and status registers through the memory-mapped bus interface. When the CPU executes a WFI instruction, it asserts the do_wfi signal, which the PMU detects to initiate a sleep mode transition and gate the CPU clock. When a wake event occurs, the PMU provides an interrupt pending register that the CPU reads to identify the wake source. The remaining on-chip components are described in the following sections.

4.1.1 Memory Subsystem

The memory subsystem consists of an external SPI flash for non-volatile firmware storage and an 8 KB on-chip SRAM for program execution, variable storage, and stack. At boot, the boot ROM copies the firmware binary from flash into SRAM. The SRAM operates synchronously, completing each read or write in a single clock cycle. Additionally, this design supports byte, half-word, and word writes through write strobe signals, enabling the sb, sh, and sw store instructions.

4.1.2 Reset Synchronizer

The reset synchronizer ensures a complete reset process across both clock domains. This design uses an asynchronous assert and a synchronous deassert approach. When the external reset is triggered, registers with asynchronous reset clear immediately regardless of clock activity, while registers with synchronous reset clear on the next clock edge. However, releasing the reset asynchronously can cause metastability if the release occurs too close to a clock edge. To prevent

this, the reset signal passes through a two-stage synchronizer, which delays the deassertion by two clock cycles and aligns the release with the clock edge. This ensures that all registers exit reset simultaneously on the same clock edge.

4.1.3 Clock Sources

The system receives two external clock inputs from off-chip crystal oscillators. The 4 MHz clock drives the CPU and memory interface during Active mode, while the 32 kHz clock provides a low-power timebase for the timeout counter during Sleep mode. The PMU controls oscillator enable signals that determine whether each clock source is active, allowing unused oscillators to be disabled during low-power states.

4.1.4 SRAM

The 8 KB SRAM provides single-cycle read/write access for program execution, variable storage, and the stack. It supports byte, half-word, and word writes via write strobe signals. In this design, the SRAM is described in behavioral Verilog and synthesized from individual flip-flops, accounting for 97.1% of the total synthesized cell area. In a production version, a foundry-supplied compiled SRAM macro would replace this, greatly reducing the memory footprint. For this reason, the SRAM was not included in the physical layout.

4.1.5 Peripheral Communication Controllers

This implementation integrates three peripheral communication controllers that enable the processor to interact with external devices: a serial peripheral interface (SPI) flash controller for firmware storage, a universal asynchronous receiver-transmitter (UART) module for serial debug output and Zigbee communication, and a general-purpose input/output (GPIO) controller for general-purpose digital I/O for future sensor connections. All peripheral controllers were designed

as minimal implementations. They follow standard communication protocols (UART serial format, SPI bus protocol) but are stripped to only the functionality required by the firmware.

The SPI flash controller connects the processor's memory bus to external flash memory, like NOR flash, which stores the application firmware. Because SPI flash transfers data serially, one bit per clock cycle, it has significantly lower throughput than SRAM. To overcome this speed limitation, the controller uses the SPI Fast Read protocol in hardware and handles command sequencing, address shifting, and data capture without involving the CPU.

The flash controller plays a key role during boot. When the system powers up, the hardware boot ROM runs a loop that reads the firmware image from flash address 0x0800_0000 using the SPI controller and writes it to SRAM at 0x2000_0000. Each 32-bit word takes about 144 clock cycles to read, which includes 8 command bits, 24 address bits, 8 dummy bits, and 32 data bits, each requiring two SPI clock edges, resulting in a total boot time of approximately 100 ms at a 4 MHz clock frequency. Once the copy process is finished, the processor begins execution from SRAM at full single-cycle throughput.

The UART controller implements the standard 8N1 asynchronous serial protocol [14, 15]. It has a 16-bit baud rate divisor that sets the bit timing based on the system clock. In this design, a divisor of 434 generates a baud rate of approximately 9,216 from the 4 MHz CPU clock, which is within 4% of the standard 9,600 baud rate. The transmitter FSM takes a byte from the CPU bus and sends it as a start bit, eight data bits (least significant bit first), and a stop bit. The receiver FSM looks for the start bit's falling edge, waits half a baud period to sample in the middle of each bit, then reads the eight data bits and checks the stop bit. Because the external RX signal is not in sync with the system clock, it goes through a double-flop synchronizer to avoid metastability [16].

The CPU uses three memory-mapped registers to access the UART: a data register for serial bytes, a status register for checking transmit/receive flags, and a control register for optional interrupts.

The GPIO controller provides eight bidirectional digital I/O pins that can be individually configured as inputs or outputs under firmware control through three memory-mapped registers: an output value register, a direction control register, and a read-only input sampling register.

4.2 Off-Chip Interface

The chip-level module exposes 24 signal pins that connect to external components on the printed circuit board. Table 4.2 summarizes all off-chip interface signals with descriptions.

Pin	Direction	Width	Description
clk_4mhz	Input	1	4 MHz crystal oscillator signal
clk_32khz	Input	1	32 kHz crystal oscillator signal
rst_n	Input	1	Active-low asynchronous reset
pir_sensor	Input	1	PIR sensor input
gpio[7:0]	Bidirectional	8	General-purpose I/O
uart0_tx	Output	1	UART TX
uart0_rx	Input	1	UART RX
flash_spi_clk	Output	1	SPI serial clock to external flash
flash_spi_cs_n	Output	1	SPI chip select
flash_spi_mosi	Output	1	SPI master-out slave-in
flash_spi_miso	Input	1	SPI master-in slave-out
debug_state[3:0]	Output	4	PMU state machine
debug_clk_cpu_en	Output	1	Clock gate enable status
debug_cpu_trap	Output	1	CPU trap/WFI status

Table 4.2: Off-chip interface signals.

Two clock input pins (clk_4mhz and clk_32khz) receive signals from external crystal oscillators. The 4 MHz crystal provides the primary system clock for the SPI flash controller and, through the PMU's clock gate, for the CPU and peripheral controllers. The 32 kHz crystal drives the PMU's real-time counter, which measures the configurable inactivity timeout during Sleep mode.

The 4 MHz system clock was chosen for this project to minimize energy consumption throughout the entire operation while maintaining compatibility with the chip's communication protocols. Since dynamic power scales linearly with frequency and quadratically with supply voltage, as shown in Equation 4.1, a lower clock frequency results in substantial energy savings.

$$P_{dynamic} = CV^2{}_{DD}f \tag{4.1}$$

where α is the switching activity factor, C is the total switched capacitance, V_{dd} is the supply voltage, and f is the clock frequency [13]. The firmware workload involves reading an interrupt register, transmitting a short Zigbee command packet via UART, and updating PMU control registers. This takes fewer than 500 clock cycles, or about 125 microseconds at 4 MHz. While a higher frequency would finish the workload faster, it would also use more dynamic power because dynamic power increases with frequency.

The 4 MHz clock also provides practical advantages for peripheral timing. For example, the UART baud rate generator divides 4 MHz by 434 to produce about 9,216 baud, which is within an acceptable error margin of 4% of the standard 9,600 baud. The divisor is set in RTL, so the baud rate can be adjusted to other standard rates by simply modifying the parameter in Verilog and re-synthesizing. The SPI flash controller creates a 2 MHz serial clock that matches standard NOR flash timing. Each UART byte takes about 1.09 ms to transmit at 9,216 baud (10 bits per frame),

which is the main time cost during Active mode. In other words, increasing clock frequency wouldn't effectively reduce the overall wake time.

The 32 kHz (actually 32.768 kHz) crystal oscillator runs the PMU's real-time counter during Sleep mode. It keeps accurate time while using very little power. This frequency is commonly used for low-power timekeeping because it is an exact power of two

(2^{15} = 32,768), allowing a simple binary counter to create a precise one-second interval and uses much less power than the 4 MHz system clock.

The SPI flash interface uses four pins: flash_spi_clk (serial clock output), flash_spi_cs_n (active-low chip select), flash_spi_mosi (data output to flash), and flash_spi_miso (data input from flash). These signals connect to an external SPI flash memory device that stores the firmware image, which is read and loaded into SRAM during system initial boot phase. The UART interface has two pins: uart0_tx for transmission and uart0_rx for reception. In this application, the pins are connected to a Zigbee end module to send or receive data packets. Note that the TX pin can be connected to a USB-to-UART adapter to monitor serial debug output in a terminal. The GPIO interface has eight bidirectional pins. Although the GPIO controller is fully built in hardware, the current firmware does not use any GPIO pins. The GPIO interface is included for future firmware updates, such as reading additional sensors or controlling status indicators.

A pir_sensor input pin provides a direct path from the PIR sensor to the PMU. This pin bypasses the GPIO controller and the CPU entirely, allowing the PMU to detect motion and initiate wake-up from Deep-sleep state even when the CPU clock is gated off. The PMU includes a double-flop synchronizer on this input to safely cross the asynchronous external signal into the 4 MHz clock domain.

An active-low reset pin (rst_n) drives the on-chip reset synchronizer, which generates a synchronized reset signal for all internal logic. Three debug output pins (debug_state[3:0], debug_clk_cpu_en, debug_cpu_trap) expose the PMU state machine, clock gate status, and CPU trap/WFI status for external observation during testing without requiring UART communication.

4.3 Firmware Development

The application firmware is written in C and cross-compiled using the RISC-V GCC toolchain (riscv32-unknown-elf-gcc) targeting the RV32E base integer instruction set with the ILP32E ABI. The RV32E variant uses 16 general-purpose registers, rather than the standard 32 (e.g., RV32I), which is sufficient for the application's control-oriented workload and reduces the total core area excluding memories by 25% in the synthesized processor [6]. The compiler is invoked with -Os optimization to minimize code size, along with -ffreestanding, -nostdlib, -nostartfiles, and -fno-builtin to eliminate all standard C library dependencies [7, 8]. Dead code elimination is enabled through -ffunction-sections/-fdata-sections at compile time and --gc-sections at link time [8]. The build process creates an ELF (.elf) executable, extracts a raw binary with objcopy, and converts it to Verilog hex format (.vhex) with the correct byte order for the processor's little-endian memory. Note that the .vhex file is what is used in pre-synthesis simulation. In hardware, the raw binary is programmed directly into the SPI flash chip using a standard flash programmer.

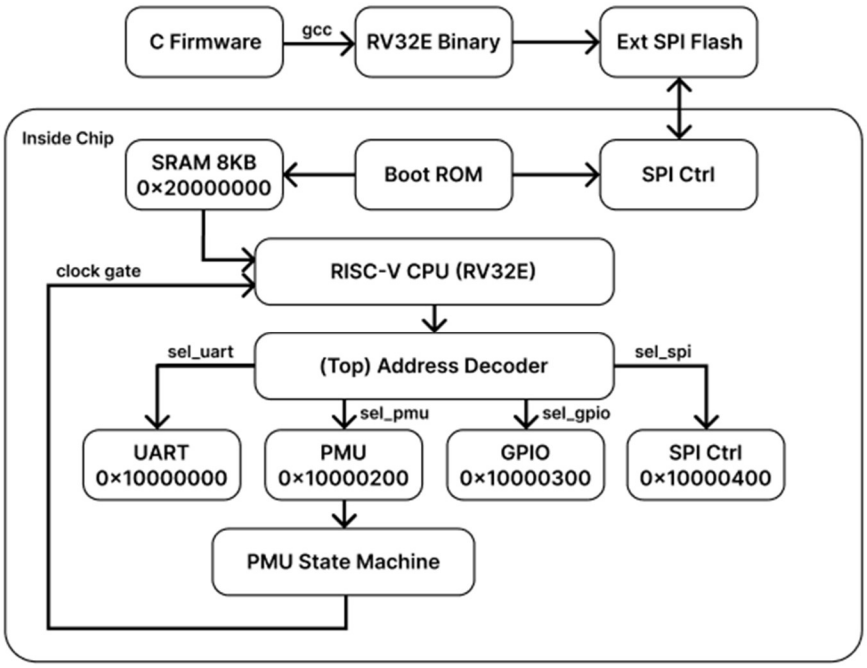


Figure 4.2: Firmware data path from compilation to hardware execution.

Figure 4.2 illustrates the firmware data path from source code to hardware execution. The compiled binary is stored in external SPI NOR flash. At power-on, the hardwired boot ROM directs the CPU to copy the firmware image from flash into on-chip SRAM through the SPI flash controller. After copying 8 KB, the CPU starts running the firmware from SRAM. While running, the firmware communicates with hardware peripherals through memory-mapped store and load instructions. For example, writing a character to address 0x10000000 leads the address decoder to assert sel_uart, sending the data to the UART controller for serial output. Writing a power state value to 0x10000200 sends the data to the PMU, which updates its state machine and controls the CPU clock gate accordingly.

The compiled firmware binary uses 716 bytes (179 words) of instruction memory and includes no initialized global variables. A custom linker script places all sections, consisting of startup code, program instructions, read-only data, and stack, in SRAM starting at 0x20000000,

since the boot ROM copies the firmware image from flash to SRAM before execution begins. A custom startup assembly file (startup.S) replaces the standard C runtime, performing only stack pointer initialization before jumping to the main function. The standard .bss clearing loop is intentionally omitted because the firmware uses only local variables and direct hardware register accesses. Even though the current binary uses less than 9% of the available memory, the 8 KB size was chosen to give developers enough space to expand the application for more complex sensing or to add more sensors without changing the hardware.

The firmware accesses all hardware peripherals via memory-mapped registers defined as C volatile pointers, preventing the compiler from optimizing away or reordering hardware accesses. Two inline assembly macros were utilized to provide access to standard RISC-V instructions: WFI() halts the CPU until an interrupt occurs, and FENCE() acts as a compiler memory barrier to ensure the compiler does not reorder the preceding PMU state register write past the subsequent WFI instruction.

Chapter 5: Verification and Simulation

The design was verified using a pre-synthesis RTL simulation to validate functional correctness, and a post-synthesis gate-level simulation to confirm that the synthesized netlist maintained the intended behavior. Both simulations covered the entire motion sensor firmware cycle, including boot, Zigbee network join, Deep-sleep entry, PIR wake-up, light control, Sleep mode timeout, another light control, and returning to Deep-sleep. The simulations use identical testbench stimulus and SPI flash models. As shown in Table 5.1, both simulation stages produced equivalent results, confirming that the RTL and synthesized netlist behave the same.

Metric	Pre-Synthesis (RTL)	Post-Synthesis (Gate-Level)
Simulator	Icarus Verilog	Synopsys VCS S-2021.09
Design under test	Behavioral RTL	TSMC 65nm netlist
Boot complete (Deep-sleep #1)	100 ms	100 ms
PIR wake → Deep-sleep #2	1 s timeout	1 s timeout
CPU clock edges (boot)	404,766	404,766
CPU clock edges (PIR cycle)	500	495

Table 5.1: Pre-synthesis and post-synthesis performance comparison.

5.1 Simulation Environment

Both simulation stages share a testbench that provides an identical test scenario. The testbench generates two independent clocks: a 4 MHz system clock and a 32.768 kHz RTC clock. An active-low reset signal is held asserted for 1 ms of simulation time, then released synchronously to the rising edge of both clocks to ensure clean initialization across all clock domains.

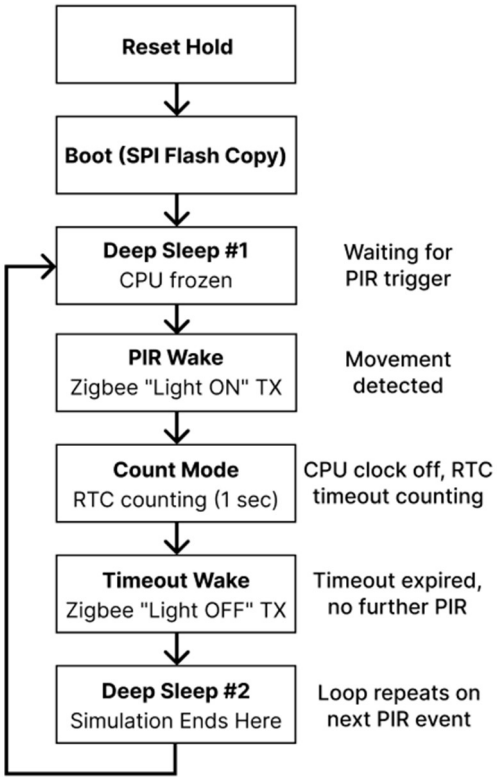


Figure 5.1: Simulation sensing cycle sequence.

Figure 5.1 illustrates the test scenario used in both simulation stages. After the initial reset hold and boot phase, the firmware enters Deep-sleep and waits for a PIR trigger. The testbench sets the pir_sensor input high for 2 ms at 120 ms after reset release to simulate motion detection. The firmware then sends a "Light ON" command, enters Sleep mode for a 1-second (customizable) RTC timeout, sends a "Light OFF" command when the timeout ends, and returns to Deep-sleep. The simulation ends automatically when it sees the second Deep-sleep entry, confirming a full sensing cycle has been completed successfully. To avoid endless simulation if something fails, there is a 2-second safety timeout after the PIR trigger and a 3-second absolute timeout.

A behavioral SPI flash model provides the firmware image to the on-chip boot ROM's copy loop. The model loads the firmware binary in Verilog hex format using the $readmemh system

task and responds to SPI Fast Read commands (opcode 0x0B), which is a standard command defined by SPI NOR flash manufacturers, by shifting out the addressed data one bit per SPI clock cycle.

The testbench includes the following monitoring infrastructure:

- a CPU clock edge counter that tracks total CPU clock rising edges throughout the simulation.
- a 2 ms (after reset release) post-reset state check that reports the clock gate latch, CPU reset, trap, WFI, and memory valid signals.
- a PMU state monitor that displays every state transition.
- a clock gating monitor that reports when the CPU clock is enabled or disabled.
- a trap monitor that reports WFI assertion and deassertion events.
- a UART monitor that records each CPU write to the UART data register with a timestamp.
- a GPIO output monitor that displays value changes on the 8-bit GPIO port.
- a progress monitor that prints the PMU state, clock gate status, trap signal, SPI activity, GPIO state, and CPU clock edge count every 10 ms.

A pre-synthesis simulation was performed using Icarus Verilog, an open-source Verilog simulator, running a zero-delay behavioral simulation of the RTL. Post-synthesis simulation was performed using Synopsys VCS (version S-2021.09), running the gate-level netlist produced by Synopsys Design Compiler with the TSMC 65nm standard cell library.

5.2 Pre-Synthesis Simulation

The pre-synthesis simulation confirmed that the full RTL design worked as expected by executing the firmware through the sensing cycle described in Section 5.1. The boot phase took about 100 ms and used 404,766 CPU clock edges. This total includes the SPI flash transfer (about 144 cycles per word for 2,048 words), the boot ROM copy loop, and firmware setup up to the first Deep-sleep entry. Figure 5.2 shows the key power state transitions captured during the simulation.



Figure 5.2: Pre-synthesis simulation waveforms during the complete sensing cycle. (a) First Deep-sleep entry after boot at 101 ms. (b) PIR-triggered wake-up from Deep-sleep at 121 ms. (c) Transition from Active to Sleep mode after transmitting the "Light ON" command. (d) Timeout-triggered wake from Sleep mode at 1,121 ms. (e) Final Deep-sleep entry after transmitting the "Light OFF" command, at which point the simulation terminates.

The UART TX monitor recorded ASCII status messages to confirm each firmware state change. At startup, it showed "BOOT". For the five Zigbee join attempts, it displayed "T1" through "T5", each followed by "F" to simulate a failure. When no network was found, it showed "NONET". After initialization, it displayed "RDY", and when entering Deep-sleep, it showed "DS". After a PIR event, the monitor displayed "PIR", "LON", and "SENT" during the light-on transmission, "CM" when entering Sleep mode, and "TO", "LOFF", "SENT", and "DS" after the timeout before returning to Deep-sleep.

During the simulation, the PMU state machine transitioned correctly through all three power states. The chip remained in Active mode during boot and firmware execution, transitioned to Deep-sleep with both clocks gated off after initialization, and entered Sleep mode when the firmware initiated the RTC timeout counter on the 32 kHz clock. The complete PIR sensing cycle required 500 clock edges. The simulation ran for a total of 1,121 ms (1,120 ms after reset release) and terminated automatically when the second Deep-sleep entry was detected. The complete pre-synthesis simulation output log, including all UART messages, PMU state transitions, and clock edge counts, is provided in Appendix A.

5.3 Synthesis

Synthesis was conducted using Synopsys Design Compiler (version S-2021.06-SP2), using the TSMC 65nm General Purpose Plus (tcbn65gplus) standard cell library. The design was synthesized across three library corners: typical (0.8V), worst (0.72V), and best (0.88V). The compile_ultra command was employed with hierarchy preservation enabled to maintain module boundaries for debugging, and logic structuring was applied to optimize both timing and area.

The DC compiler reported that the synthesized logic, consisting of the CPU, PMU, peripheral controllers, and boot ROM, occupies 26,583 μm² (15,760 μm² combinational and 10,823 μm² sequential), excluding the 8 KB SRAM which would be replaced by a foundry-provided compiled macro in a production design.

Clock constraints defined a 250 ns period (4 MHz) for the primary system clock and a 30,518 ns period (32 kHz) for the RTC clock. The two clock domains were declared as asynchronous using set_clock_groups to exclude false cross-domain timing paths.

5.4 Post-Synthesis Simulation

The post-synthesis gate-level simulation verifies that the synthesized netlist maintains the same functional behavior as the original RTL by running the same firmware and testbench stimulus on the TSMC 65nm standard cell netlist generated with Design Compiler.

The design initially had X-propagation issues because the CPU used a synchronous reset, while the TSMC cell library models flip-flops with Verilog UDPs. In behavioral simulation, synchronous reset sets all registers to known values on the first clock edge. However, UDP-based flip-flops start in the X state and only clear through an asynchronous reset pin on the physical cell. This caused a deadlock: the CPU's trap and do_wfi outputs were stuck at X, which affected the PMU's clock gate enable signal and held clk_cpu at X, so the CPU could not run its own synchronous reset. The problem was fixed by changing the instruction decoder to use an asynchronous reset and adding a reset timing margin in the PMU, giving all CPU outputs enough clock cycles to reach valid logic levels before the clock-gate logic responds.

Consequently, the post-synthesis simulation completed the full sensing cycle without any X propagation. The boot ROM copied firmware into SRAM over 404,766 CPU clock edges by the

101 ms mark (100 ms after the reset release at 1 ms). The firmware ran and entered Deep-sleep, and the PIR stimulus at 121 ms triggered the full wake, transmit, timeout, transmit, and sleep cycle, using 495 more clock edges. The post-synthesis simulation results matched the pre-synthesis results, confirming that the RTL and the synthesized netlist are functionally equivalent. The complete post-synthesis simulation output log is provided in Appendix B.

Chapter 6: Physical Implementation and Results

6.1 Physical Layout

The physical layout was generated using Synopsys IC Compiler II from the synthesized gate-level netlist produced in Section 5.3. The ICC2 place-and-route flow was used with TSMC 65nm library. The TCL scripts, including design setup, floorplanning, power grid generation, and optimization, were utilized to convert the synthesized design (top module including all components except SRAM) to layout. The floorplan was resized for the larger cell count. The 8 KB SRAM was excluded from the layout as a black box since it would be replaced by a compiled macro in a production flow.

Metric	Value
Technology	TSMC 65nm GP + tcbn65gplus_pwr
Layout tool	Synopsys ICC2 S-2021.06-SP2
Standard cell count	14,155
Cell area	47,136.6 μm²
Die area dimensions	70,862 μm² (266.8 × 265.6 μm)
Core area	65,638 μm² (256.8 × 255.6 μm)
Utilization	71.8%
Total I/O pins	24 (described in Table 4.2)

Table 6.1: Physical layout summary.

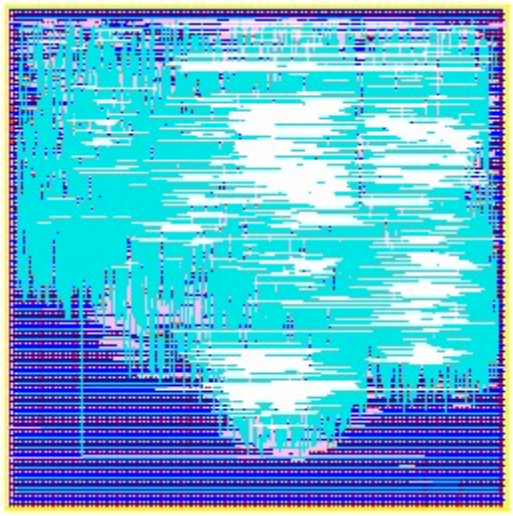


Figure 6.1: Layout of designed MCU (excluding SRAM).

The ICC2 flow uses four main optimization steps: floorplan setup with power grid generation, timing-driven placement, clock tree synthesis for both the 4 MHz and 32 kHz clock domains, and routing optimization. After place and route, the final layout has 14,155 standard cells and covers 47,137 μm² in a 256.8 × 255.6 μm core area (0.066 mm²) at 71.8% utilization. The area increase from 26,583 μm² (reported by Design Compiler) to 47,137 μm² is due to buffer cells, hold-fix cells, and tie cells added by ICC2 during optimization. Table 6.1 summarizes these results, and Figure 6.1 shows the layout.

6.2 Power Analysis

Power consumption was estimated using Synopsys Design Compiler's power analysis under worst-case operating conditions (0.72 V, 25°C) with the TSMC 65nm standard cell library. Each state was analyzed with the modules that remain active in that state, as summarized in Table 6.2.

Metric	Active mode	Sleep mode	Deep-sleep mode
Modules included	CPU, PMU, top, timeout counter, SRAM 8kB, all peripheral controllers, boot ROM	PMU, top, timeout counter	PMU, top, timeout counter
Active clock	4 MHz	32.768 kHz	None
Internal power	25.39 µW	30.37 nW	0.00 µW
Switching power	367.58 nW	1.43 nW	0.00 µW
Dynamic power	25.75 µW	31.80 nW	0.00 µW
Leakage power	46.71 µW	7.62 µW	7.50 µW
Total power	72.46 µW	7.65 µW	7.50 µW

Table 6.2: Power analysis results of three different PMU states.

In Active mode, the total power consumption is 72.46 µW, of which leakage accounts for 64% and dynamic power accounts for 36%. The Active mode estimate includes the 8 KB SRAM synthesized from flip-flops, which contributes significantly to a large leakage area. A real production design using a compiled SRAM macro would reduce this leakage.

For both sleep mode analyses, the flip-flop SRAM was excluded, as a compiled SRAM macro would be used in production. Only the PMU, top-level glue logic, and timeout counter were included in the analysis.

In Sleep mode, the CPU clock is gated off, and only the PMU and timeout counter operate on the 32 kHz RTC clock, consuming 7.65 µW. Dynamic power decreases to 31.80 nW because only the low-frequency RTC logic remains active, while leakage drops to 7.62 µW due to clock-gating of the CPU and peripherals.

In Deep-sleep mode, both clock domains are disabled, and power consumption is entirely leakage at 7.50 µW. Since the DC compiler requires a clock reference for analysis, a 1 Hz clock was applied to obtain the leakage estimate. The 0.15 µW difference between Sleep and Deep-sleep modes represents the RTC switching activity eliminated when the 32 kHz oscillator is turned off.

Metric	This Design	STM32U5
Type	MCU	MCU
Process	65 nm GP	40 nm ULP
Core	Customized RV32E	Arm® 32-bit Cortex®-M33 CPU with TrustZone®, MPU, DSP, and FPU
Active mode power @ 4 MHz	72.46 µW (@ 0.72 V)	231.12 µW (32.1 µA/MHz @ 1.8 V)
Deep-sleep / Shutdown power	7.50 µW (@ 0.72 V)	160 nA (288 nW @ 1.8 V)
Supply voltage range	0.72 ~ 0.88 V	1.71 ~ 3.6 V

Table 6.3: Power comparison with commercial ULP MCU.

Table 6.3 compares the power consumption of the designed MCU against the STM32U5 in Active mode and Deep-sleep mode, positioning the design's power performance against an established commercial ultra-low-power reference.

The Active mode total power of 72.46 µW is approximately 31% of the STM32U5's 231.12 µW at 4 MHz. This lower power is attributed to the combination of a reduced supply voltage (0.72V vs 1.8V), which lowers dynamic power quadratically, and the minimal architectural

footprint without pipeline, hardware multiplier, or DSP extensions, which reduces both switching activity and transistor count.

The Deep-sleep power of 7.50 μW is significantly higher than the STM32U5's shutdown mode power of 0.29 μW. This large gap has two main causes. First, the TSMC 65nm GP standard cell library used in this design is optimized for performance rather than standby current. An ultra-low-leakage (ULL) or low-power (LP) library variant would substantially reduce static power, but neither was available in the design environment. Second, the PMU and timeout counter modules are not power-gated during Deep-sleep. Although both clocks are disabled and no switching occurs, the transistors in these modules continue to leak. Implementing power gating for these modules would require additional design complexity, including isolation cells, multiple power supply voltages, and retention registers, which was beyond the scope of this work.

6.3 Timing Analysis

Timing analysis was performed using the pre-layout synthesized netlist and Design Compiler static timing analysis. The design meets all setup timing constraints at both the 4 MHz system clock (250 ns period) and the 32.768 kHz RTC clock (30,518 ns period), with positive slack on all paths. The only timing violations reported are hold-time violations on direct flip-flop-to-flip-flop paths, with slack values between -0.11 ns and -0.02 ns.

These hold-time violations happen because Synopsys Design Compiler uses ideal clocks with no interconnect delay between cells at the pre-layout stage. The worst violation, -0.11 ns, appears on the clock gating latch-to-AND-gate path. Other violations are found on shift register stages, CDC double-flop synchronizers, and CPU internal registers. In standard ASIC design, these

hold violations are usually fixed during place and route by adding buffers and performing clock tree synthesis [17].

The boot time takes approximately 100 ms, mainly due to the SPI flash serial transfer. Each 32-bit word takes 144 clock cycles using the SPI Fast Read protocol. After booting, the firmware runs from SRAM with single-cycle throughput. The PIR sensing cycle after synthesis takes 495 clock edges, which includes both wake events, both Zigbee transmissions, and all PMU register accesses. This adds up to about 124 μs of total CPU active time per sensing cycle. Most of the active time per wake is spent on UART serial transmission at 9,214 baud (1.09 ms per byte), not on instruction execution.

6.4 Discussion

This work presents a complete RISC-V RV32E motion-sensor IoT chip that includes a 32-bit processor, a custom PMU, an SPI flash controller, a UART controller, a GPIO controller, and a boot ROM. The chip was designed and synthesized using the TSMC 65nm process, verified with both pre- and post-synthesis simulations, and runs a full motion-sensing application with Zigbee communication.

In Active mode at 4 MHz, the chip draws 72.46 μW, much lower than the STM32U5's 231.12 μW at the same frequency. This is mainly because of the lower supply voltage (0.72V instead of 1.8V) and a simpler design that omits features not needed for the target application, such as hardware multiplication, division, pipelining, and DSP extensions. In Deep-sleep mode, the chip uses 7.5 μW, which is higher than the sub-μW levels of some commercial ultra-low-power microcontrollers [12, 18], primarily due to the leakage characteristics of the TSMC 65nm general-purpose standard cell library, which is optimized for performance rather than low standby current.

Even assuming frequent usage of one PIR detection event every 15 minutes with a 5-second timeout, the average current consumption is 10.42 μA at 0.72 V, dominated almost entirely by Deep-sleep leakage. With a CR2032 coin cell (225 mAh), battery life is estimated at approximately 2.5 years; with two AA batteries (2,500 mAh), approximately 27 years, though battery self-discharge would limit practical lifetime beyond 10 years. The negligible contribution of Active mode power to average consumption confirms that the clock gating strategy is effective, and further battery life improvement depends primarily on reducing Deep-sleep leakage.

Chapter 7: Conclusion and Future Work

7.1 Summary of Work

The goal of this work was to design and verify a complete RISC-V RV32E motion sensor IoT chip with integrated power management on the TSMC 65nm process. The chip uses three power states and relies mainly on clock gating to save power. Simulations before and after synthesis demonstrated that the chip worked correctly throughout the sensing cycle. In Active mode, it used 72.46 µW, which is significantly lower than the STM32U5's 231.12 µW at the equivalent frequency (4 MHz).

Unlike most academic RISC-V ASIC projects that focus on processor core optimization and validate with generic benchmarks, this work implements a complete application-specific SoC driven by customizable firmware that executes a real motion sensing application with sleep/wake management. The implemented architecture allows the sensing cycle, timeout, communication protocol, and sensor type to be modified through software changes alone, and the memory-mapped peripheral bus enables additional sensors or communication modules to be integrated with minimal hardware changes. The complete RTL, firmware, synthesis scripts, and verification testbenches provide a reusable platform for low-power IoT sensor nodes, offering opportunities for further research in academia and ASIC product development in industry.

7.2 Future Enhancements

With only one application-specific testbench scenario and limited development time, the current design has various opportunities for improvement as follows:

- The 8 KB SRAM, which accounts for 97.1% of the total cell area when synthesized from flip-flops, would be replaced with a foundry-provided compiled SRAM macro in a

production design, significantly reducing both die area and Deep-sleep leakage. Additionally, the SRAM and CPU domains could be power-gated during Deep-sleep to achieve lower standby current, at the cost of requiring a full firmware reload from SPI flash on each wake event. Evaluating this tradeoff would require further power analysis comparing the leakage savings of power gating against the energy cost of the 100 ms reboot sequence.

- The PMU and timeout counter modules remain ungated during Deep-sleep, contributing to the 7.5 μW of Deep-sleep leakage. Power gating non-essential PMU logic and timeout counter could reduce it significantly. Furthermore, extracting the PIR wake detection logic into a smaller dedicated sub-module (separate from the current PMU) that operates independently during Deep-sleep would reduce the amount of active circuitry and leakage.
- At the physical implementation level, SDF-annotated gate-level simulation with back-annotated parasitics from place-and-route would provide more accurate timing verification beyond the zero-delay simulation performed in this work. Additionally, targeting an ULL process library instead of the general-purpose library used in this work could significantly reduce the static power consumption of the chip.

Ultimately, fabrication and silicon characterization would validate the simulation-based power and timing estimates presented in this work.

Appendix A: Pre-Synthesis Simulation Output

PS C:\Thesis\Design\publishment\works\pre_synthesis_sim> vvp
motion_sensor_sim

WARNING: testbench_pre_synth_uart.v:322: $readmemh(motion_sensor_minimal.vhex): Not enough words in the file for the requested range [0:16383].

==========

SPI Flash Model Loaded

Flash[0] = 20002137

Flash[1] = ffc10113

VCD info: dumpfile pre_synth_sim.vcd opened for output.

==========

MOTION SENSOR IOT CHIP, PRE-SYNTHESIS SIMULATION

UART Zigbee Firmware

Auto-Stop After One Complete Cycle

==========

[0 ms] CPU Clock Gate: ENABLED

[0 ms] PMU -> ACTIVE

[CLK_CPU] Edge #1 @ 125000

[CLK_CPU] Edge #2 @ 375000

[CLK_CPU] Edge #3 @ 625000

[CLK_CPU] Edge #4 @ 875000

[CLK_CPU] Edge #5 @ 1125000

[CLK_CPU] Edge #1000 @ 249875000

[CLK_CPU] Edge #2000 @ 499875000

[CLK_CPU] Edge #3000 @ 749875000

[CLK_CPU] Edge #4000 @ 999875000

[1000 ms] Releasing reset...

[1000 ms] CPU reset hold expired

[POST-RESET +500ns] debug_state=0011 gate=1 trap=0 spi_cs=1

[POST-RESET +5us] debug_state=0011 gate=1 trap=0 spi_cs=1

[POST-RESET +130us] debug_state=0011 gate=1 trap=0 spi_cs=0

[CLK_CPU] Edge #5000 @ 1249875000

[CLK_CPU] Edge #6000 @ 1499875000

[CLK_CPU] Edge #7000 @ 1749875000

[CLK_CPU] Edge #8000 @ 1999875000

[CLK_CPU] Edge #9000 @ 2249875000

[CLK_CPU] Edge #10000 @ 2499875000

==========

2ms POST-RESET STATE CHECK

clk_cpu edges: 12090

clk_gate_latch: 1

clk_cpu: 1

cpu_reset: 0

cpu_trap: 0

cpu_do_wfi: 0

cpu_mem_valid_raw: 1

debug_state: 0011

debug_clk_cpu_en: 1

flash_spi_cs_n: 0

==========

[11000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=44090 gpio=00000000

…

[91000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=364090 gpio=00000000

[100000 ms] UART BUS WRITE: 'B' (0x42)

[100000 ms] UART BUS WRITE: 'O' (0x4f)

[100000 ms] UART BUS WRITE: 'O' (0x4f)

[100000 ms] UART BUS WRITE: 'T' (0x54)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'T' (0x54)

[100000 ms] UART BUS WRITE: '1' (0x31)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'F' (0x46)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'T' (0x54)

[100000 ms] UART BUS WRITE: '2' (0x32)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'F' (0x46)

[100000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: '3' (0x33)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=404090 gpio=00000000

[101000 ms] UART BUS WRITE: 'F' (0x46)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: '4' (0x34)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'F' (0x46)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: '5' (0x35)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'F' (0x46)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'N' (0x4e)

[101000 ms] UART BUS WRITE: 'O' (0x4f)

[101000 ms] UART BUS WRITE: 'N' (0x4e)

[101000 ms] UART BUS WRITE: 'E' (0x45)

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'R' (0x52)

[101000 ms] UART BUS WRITE: 'D' (0x44)

[101000 ms] UART BUS WRITE: 'Y' (0x59)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'D' (0x44)

[101000 ms] UART BUS WRITE: 'S' (0x53)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] CPU TRAP/WFI asserted

[101000 ms] CPU Clock Gate: DISABLED

```
[101000 ms] PMU -> DEEP_SLEEP
=========================================================
 DEEP SLEEP ENTRY #1 @ 101000 ms
=========================================================
   [111000 ms] debug_state=0000  gate=0  trap=1  spi_cs=1  spi_clk=0  clk_edges=404766  gpio=00000000
[121000 ms] debug_state=0000  gate=0  trap=1  spi_cs=1  spi_clk=0  clk_edges=404766  gpio=00000000
=========================================================
 Triggering PIR @ 121000 ms
   PMU State: 00  Clock Gate: 0
=========================================================
[121000 ms] GPIO = 0x01
=========================================================
 [121000 ms] PIR TRIGGERED
   PMU State: 00  Clock Gate: 0  Trap: 1
=========================================================
[121000 ms] CPU Clock Gate: ENABLED
[121000 ms] PMU -> ACTIVE
[121000 ms] CPU TRAP/WFI deasserted
[121000 ms] UART BUS WRITE: 'P' (0x50)
[121000 ms] UART BUS WRITE: 'I' (0x49)
[121000 ms] UART BUS WRITE: 'R' (0x52)
[121000 ms] UART BUS WRITE: [0x0a]
[121000 ms] UART BUS WRITE: 'L' (0x4c)
[121000 ms] UART BUS WRITE: 'O' (0x4f)
[121000 ms] UART BUS WRITE: 'N' (0x4e)
[121000 ms] UART BUS WRITE: [0x0a]
[121000 ms] UART BUS WRITE: 'S' (0x53)
[121000 ms] UART BUS WRITE: 'E' (0x45)
[121000 ms] UART BUS WRITE: 'N' (0x4e)
[121000 ms] UART BUS WRITE: 'T' (0x54)
[121000 ms] UART BUS WRITE: [0x0a]
[121000 ms] UART BUS WRITE: 'C' (0x43)
[121000 ms] UART BUS WRITE: 'M' (0x4d)
```

[121000 ms] UART BUS WRITE: [0x0a]

[121000 ms] CPU TRAP/WFI asserted

[121000 ms] CPU Clock Gate: DISABLED

[121000 ms] PMU -> SLEEP

[123000 ms] GPIO = 0x00

[131000 ms] debug_state=0001 gate=0 trap=1 spi_cs=1 spi_clk=0 clk_edges=405266 gpio=00000000

…

[1121000 ms] debug_state=0001 gate=0 trap=1 spi_cs=1 spi_clk=0 clk_edges=405266 gpio=00000000

[1121000 ms] CPU Clock Gate: ENABLED

[1121000 ms] PMU -> ACTIVE

[1121000 ms] CPU TRAP/WFI deasserted

[1121000 ms] UART BUS WRITE: 'T' (0x54)

[1121000 ms] UART BUS WRITE: 'O' (0x4f)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] UART BUS WRITE: 'L' (0x4c)

[1121000 ms] UART BUS WRITE: 'O' (0x4f)

[1121000 ms] UART BUS WRITE: 'F' (0x46)

[1121000 ms] UART BUS WRITE: 'F' (0x46)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] UART BUS WRITE: 'S' (0x53)

[1121000 ms] UART BUS WRITE: 'E' (0x45)

[1121000 ms] UART BUS WRITE: 'N' (0x4e)

[1121000 ms] UART BUS WRITE: 'T' (0x54)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] UART BUS WRITE: 'D' (0x44)

[1121000 ms] UART BUS WRITE: 'S' (0x53)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] CPU TRAP/WFI asserted

[1121000 ms] CPU Clock Gate: DISABLED

[1121000 ms] PMU -> DEEP_SLEEP

==========

DEEP SLEEP ENTRY #2 @ 1121000 ms

==========

```
==========================================================
        SIMULATION COMPLETE
     One Complete Cycle Executed
==========================================================

testbench_pre_synth_uart.v:113: $finish called at 1121399875000 (1ps)
```

Appendix B: Post-Synthesis Simulation Output

```
Chronologic VCS simulator copyright 1991-2021
Contains Synopsys proprietary information.
Compiler version S-2021.09; Runtime version S-2021.09;  May 14 10:22 2026
=========================================================
SPI Flash Model Loaded
Flash[0] = 20002137
Flash[1] = ffc10113
=========================================================
 MOTION SENSOR IOT CHIP, POST-SYNTHESIS SIMULATION
 UART Zigbee Firmware
 Auto-Stop After One Complete Cycle
=========================================================
[0 ms] CPU Clock Gate: ENABLED
[0 ms] PMU -> ACTIVE
[CLK_CPU] Edge #1 @ 125000
[CLK_CPU] Edge #2 @ 375000
[CLK_CPU] Edge #3 @ 625000
[CLK_CPU] Edge #4 @ 875000
[CLK_CPU] Edge #5 @ 1125000
[CLK_CPU] Edge #1000 @ 249875000
[CLK_CPU] Edge #2000 @ 499875000
[CLK_CPU] Edge #3000 @ 749875000
[CLK_CPU] Edge #4000 @ 999875000
[1000 ms] Releasing reset...
[1000 ms] CPU reset hold expired
[POST-RESET +500ns] debug_state=0011 gate=1 trap=0 spi_cs=1 clk_cpu=0 latch=1
[POST-RESET +5us] debug_state=0011 gate=1 trap=0 spi_cs=1 clk_cpu=0 latch=1
[POST-RESET +130us] debug_state=0011 gate=1 trap=0 spi_cs=0 clk_cpu=0 latch=1
[CLK_CPU] Edge #5000 @ 1249875000
[CLK_CPU] Edge #6000 @ 1499875000
[CLK_CPU] Edge #7000 @ 1749875000
```

[CLK_CPU] Edge #8000 @ 1999875000

[CLK_CPU] Edge #9000 @ 2249875000

[CLK_CPU] Edge #10000 @ 2499875000

==========

2ms POST-RESET STATE CHECK

clk_cpu edges: 12090

clk_gate_latch: 1

clk_cpu: 1

cpu_mem_valid_raw: 1

debug_state: 0011

debug_clk_cpu_en: 1

debug_cpu_trap: 0

flash_spi_cs_n: 0

==========

[11000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=44090 gpio=00000000

[21000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=84090 gpio=00000000

[31000 ms] debug_state=0011 gate=1 trap=0 spi_cs=0 spi_clk=0 clk_edges=124090 gpio=00000000

…

[81000 ms] debug_state=0011 gate=1 trap=0 spi_cs=0 spi_clk=1 clk_edges=324090 gpio=00000000

[91000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=364090 gpio=00000000

[100000 ms] UART BUS WRITE: 'B' (0x42)

[100000 ms] UART BUS WRITE: 'O' (0x4f)

[100000 ms] UART BUS WRITE: 'O' (0x4f)

[100000 ms] UART BUS WRITE: 'T' (0x54)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'T' (0x54)

[100000 ms] UART BUS WRITE: '1' (0x31)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'F' (0x46)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'T' (0x54)

[100000 ms] UART BUS WRITE: '2' (0x32)

[100000 ms] UART BUS WRITE: [0x0a]

[100000 ms] UART BUS WRITE: 'F' (0x46)

[100000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: '3' (0x33)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] debug_state=0011 gate=1 trap=0 spi_cs=1 spi_clk=0 clk_edges=404090 gpio=00000000

[101000 ms] UART BUS WRITE: 'F' (0x46)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: '4' (0x34)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'F' (0x46)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: '5' (0x35)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'F' (0x46)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'N' (0x4e)

[101000 ms] UART BUS WRITE: 'O' (0x4f)

[101000 ms] UART BUS WRITE: 'N' (0x4e)

[101000 ms] UART BUS WRITE: 'E' (0x45)

[101000 ms] UART BUS WRITE: 'T' (0x54)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'R' (0x52)

[101000 ms] UART BUS WRITE: 'D' (0x44)

[101000 ms] UART BUS WRITE: 'Y' (0x59)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] UART BUS WRITE: 'D' (0x44)

[101000 ms] UART BUS WRITE: 'S' (0x53)

[101000 ms] UART BUS WRITE: [0x0a]

[101000 ms] CPU TRAP/WFI asserted

[101000 ms] CPU Clock Gate: DISABLED

[101000 ms] PMU -> IDLE

==========

DEEP SLEEP ENTRY #1 @ 101000 ms

==========

[101000 ms] PMU -> DEEP_SLEEP

[102 ms] UART TX: [0xff]

[111000 ms] debug_state=0000 gate=0 trap=1 spi_cs=1 spi_clk=0 clk_edges=404766 gpio=00000000

[121000 ms] debug_state=0000 gate=0 trap=1 spi_cs=1 spi_clk=0 clk_edges=404766 gpio=00000000

==========

Triggering PIR @ 121000 ms

PMU State: 00 Clock Gate: 0

==========

==========

[121000 ms] PIR TRIGGERED

PMU State: 00 Clock Gate: 0 Trap: 1

==========

[121000 ms] GPIO = 0x01

[121000 ms] CPU Clock Gate: ENABLED

[121000 ms] PMU -> SLEEP

[121000 ms] PMU -> ACTIVE

[121000 ms] CPU TRAP/WFI deasserted

[121000 ms] UART BUS WRITE: 'P' (0x50)

[121000 ms] UART BUS WRITE: 'I' (0x49)

[121000 ms] UART BUS WRITE: 'R' (0x52)

[121000 ms] UART BUS WRITE: [0x0a]

[121000 ms] UART BUS WRITE: 'L' (0x4c)

[121000 ms] UART BUS WRITE: 'O' (0x4f)

[121000 ms] UART BUS WRITE: 'N' (0x4e)

[121000 ms] UART BUS WRITE: [0x0a]

[121000 ms] UART BUS WRITE: 'S' (0x53)

[121000 ms] UART BUS WRITE: 'E' (0x45)

[121000 ms] UART BUS WRITE: 'N' (0x4e)

[121000 ms] UART BUS WRITE: 'T' (0x54)

[121000 ms] UART BUS WRITE: [0x0a]

[121000 ms] UART BUS WRITE: 'C' (0x43)

[121000 ms] UART BUS WRITE: 'M' (0x4d)

[121000 ms] UART BUS WRITE: [0x0a]

[121000 ms] CPU TRAP/WFI asserted

[121000 ms] CPU Clock Gate: DISABLED

[121000 ms] PMU -> SLEEP

[123000 ms] GPIO = 0x00

[131000 ms] debug_state=0001 gate=0 trap=1 spi_cs=1 spi_clk=0 clk_edges=405261 gpio=00000000

…

[1121000 ms] debug_state=0001 gate=0 trap=1 spi_cs=1 spi_clk=0 clk_edges=405261 gpio=00000000

[1121000 ms] CPU Clock Gate: ENABLED

[1121000 ms] PMU -> ACTIVE

[1121000 ms] CPU TRAP/WFI deasserted

[1121000 ms] UART BUS WRITE: 'T' (0x54)

[1121000 ms] UART BUS WRITE: 'O' (0x4f)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] UART BUS WRITE: 'L' (0x4c)

[1121000 ms] UART BUS WRITE: 'O' (0x4f)

[1121000 ms] UART BUS WRITE: 'F' (0x46)

[1121000 ms] UART BUS WRITE: 'F' (0x46)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] UART BUS WRITE: 'S' (0x53)

[1121000 ms] UART BUS WRITE: 'E' (0x45)

[1121000 ms] UART BUS WRITE: 'N' (0x4e)

[1121000 ms] UART BUS WRITE: 'T' (0x54)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] UART BUS WRITE: 'D' (0x44)

[1121000 ms] UART BUS WRITE: 'S' (0x53)

[1121000 ms] UART BUS WRITE: [0x0a]

[1121000 ms] CPU TRAP/WFI asserted

[1121000 ms] CPU Clock Gate: DISABLED

[1121000 ms] PMU -> IDLE

```
==========================================================
 DEEP SLEEP ENTRY #2 @ 1121000 ms
==========================================================

==========================================================
        SIMULATION COMPLETE
     One Complete Cycle Executed
==========================================================
$finish called from file "testbench_post_synth_uart.v", line 112.
$finish at simulation time         1121399875000
          V C S   S i m u l a t i o n   R e p o r t
Time: 1121399875000 ps
CPU Time:     30.060 seconds;       Data structure size:  24.1Mb
Thu May 14 10:22:43 2026
```

*The post-synthesis simulation produces the same UART byte sequence at the same timestamps as pre-synthesis, including the boot messages, the PIR-triggered light-on transmission, and the timeout light-off transmission. UART byte writes are captured by monitoring the CPU's memory bus transactions. Functional equivalence between RTL and the synthesized gate-level netlist is confirmed by matching UART output, PMU state transitions, and simulation timing.

Bibliography

[1] IoT Analytics, "State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally," IoT Analytics, Sep. 2024. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/

[2] S. K. Jayabalan *et al.*, "Experimental optimization of an IoT-driven load adaptive wireless charging system for electric vehicles using response surface methodology," *Sci. Rep.*, vol. 15, no. 1, Art. no. 43530, Dec. 2024, pp. 2, 3, 7–9, doi: 10.1038/s41598-025-27484-x.

[3] K. Elgazzar *et al.*, "Revisiting the Internet of Things: New trends, opportunities and grand challenges," *Front. Internet Things*, vol. 1, Art. no. 1073780, 2022, pp. 1, 2, 5–7, doi: 10.3389/friot.2022.1073780.

[4] Disruptive Technologies Research AS, "Battery life: How we estimate temperature sensors can live for up to 15 years," White Paper, Rev. 1.0, Apr. 2023. [Online]. Available: https://www.disruptive-technologies.com

[5] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," EECS Dept., Univ. California, Berkeley, Tech. Rep. UCB/EECS-2014-146, 2014.

[6] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V International, 2019, pp. 17, 40. [Online]. Available: https://riscv.org/specifications/

[7] RISC-V Collaboration, "RISC-V GNU Compiler Toolchain," GitHub, 2024. [Online]. Available: https://github.com/riscv-collab/riscv-gnu-toolchain

[8] Free Software Foundation, "Using the GNU Compiler Collection (GCC)," 2024. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc.pdf

[9] C. Wolf, "PicoRV32 — A size-optimized RISC-V CPU," GitHub, 2015. [Online]. Available: https://github.com/YosysHQ/picorv32.

[10] Arm Ltd., "Arm Cortex-M0+ Processor Datasheet," Document No. ARM DDI 0484C, Rev. r0p1, 2012. [Online]. Available: https://developer.arm.com/documentation/ddi0484/latest

[11] Espressif Systems, "ESP32-S3 Series Datasheet," Version 2.1, 2024, pp. 2, 3, 37, 61–63, 68. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

[12] STMicroelectronics, "STM32U5 Series Ultra-Low-Power Microcontrollers Datasheet," Doc. No. DS13639, 2024, pp. 1, 2, 33–47, 51, 57, 58, 197. [Online]. Available: https://www.st.com

[13] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011, pp. 43, 412–415.

[14] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Hoboken, NJ, USA: John Wiley & Sons, 2008, ch. 8.

[15] J. Valvano, "Real-time Bluetooth networks," EE445L Course Notes, Ch. 3, Univ. Texas at Austin, 2024. [Online]. Available: https://users.ece.utexas.edu/~valvano/EE445L/ebook/index.htm

[16] C. E. Cummings, "Synthesis and scripting techniques for designing multi-asynchronous clock designs," in *Proc. Synopsys Users Group (SNUG)*, San Jose, CA, USA, 2001.

[17] R. Donthi, "Low power ASIC design of RISC core using 90nm technology," M.S. thesis, Dept. Elect. Eng., California State Univ., Northridge, CA, USA, Dec. 2008. [Online]. Available: https://scholarworks.calstate.edu/downloads/7w62fd069

[18] Texas Instruments, "Clock gating," Application Note SLAY015A, 2024, p. 1. [Online]. Available: https://www.ti.com/lit/wp/slay015a/slay015a.pdf